Skill Misfit after Injection: Revising and Withdrawing Retrieved Skills for LLM Agents

Anonymous ACL submission

Abstract

LLM agents increasingly load *skills*: short procedures retrieved for a task and injected into the prompt. Retrieval reads only the task description, so a skill can fit the goal yet mislead the agent as the episode unfolds. With a reasoning agent on ALFWorld, restoring the same states shows that the retrieved skills alone can cause failures: from states of games the agent wins without skills but loses with them, continuing with the skills fails in all 18 runs, whereas withdrawing them wins 11. We propose TRIM, a training-free method that scores how much the skills raise the likelihood of the agent's drafted response, reasoning included, and intervenes when they drive it: at the first firing an editor revises the skills, and if they drive the agent again they are withdrawn for the rest of the episode. With Qwen3-8B in thinking mode, TRIM wins 75, 71 and 65 of the 134 unseen test games in three runs, on average 10.3 more than the retrieved skills (7.7 points, 95% CI [2.0, 13.7]). It also leads with Qwen3-14B and on SRA-Bench ToolQA and beats the same two stages applied at injection or at random steps. On development games, withdrawing the skills helps in episodes where the signal fires repeatedly and hurts where it fires once, so skills should be withdrawn selectively rather than dropped.

1 Introduction

Agent skills package procedures, tool use and applicability conditions into short texts that an agent can reuse across tasks (Anthropic, 2025; Li et al., 2026c; Xia et al., 2026). Because a skill is written once for many tasks, whether it fits a particular run is a separate question.

Skills do not always help: a skill can lead the agent into a wrong action, add cost without changing the outcome, or leave a failure unsolved, a mismatch between the skill and its execution (Dong et al., 2026; Han et al., 2026; Wang et al., 2026f; Yu et al., 2026a; Li et al., 2026b). Misfit follows from how skills are used: they are written before the task is seen, and retrieving several from the task description alone brings in items that do not fit.

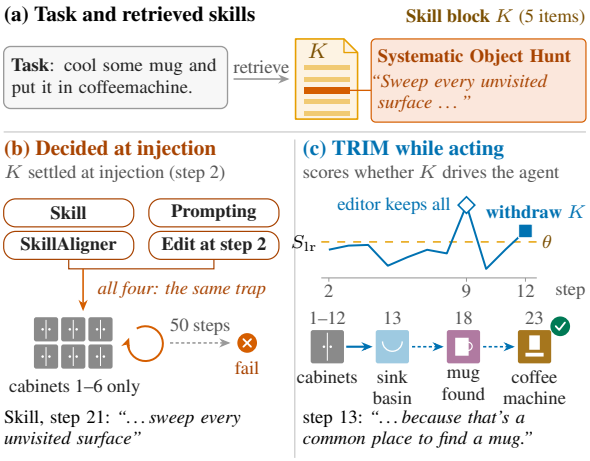


Figure 1: One test game (App. F). (a) The block retrieved from the goal holds a search routine. (b) Arms that settle the skills at injection stay in cabinets 1–6 for 50 steps. (c) TRIM has Skill's prompts up to step 12; S_{lr} fires at step 9 (the editor keeps all items) and at step 12 (block withdrawn), after which the agent leaves the cabinets and finds the mug. The agent without skills also finds it (32 steps). Icons in (c) skip steps.

Existing remedies decide how a skill is used before or at injection: when to invoke it (Chen et al., 2026b), which items to retrieve for a plan (Wang et al., 2026a), or how to rewrite the library or the retrieved block (Yu et al., 2026a; Li et al., 2026b); other work refines skills between episodes (Zhang et al., 2026c; Ouyang et al., 2026; Liu et al., 2026b). They leave open what an injected skill does as the task unfolds (the concurrent EvoSkill-GUI revises skills across repeated attempts; Chen et al., 2026a).

An agent that acts in an environment keeps receiving observations that the task description does not contain, and a skill can go wrong exactly there. In Figure 1, the block retrieved for cooling a mug holds a search routine; with it, the agent and every

method that settles the skills at injection search cabinets 1–6 for all 50 steps, whereas the agent without skills finds the mug. Revising the block before acting does not prevent this: an edit at step 2 keeps the search item, and the SkillAligner guide written before acting also ends in cabinets 1–6. Only the unfolding episode shows that the search routine has become a trap, so skills need to be revisited after observations.

We therefore target misfits that the injected skills cause after injection. TRIM asks whether the skills drive the agent: it scores the drafted response, reasoning included, with and without the skill block, and intervenes when this log-likelihood ratio, S_{lr}, reaches a threshold (Figure 2b). At the first firing an editor keeps, withholds or rewrites the items and the step is regenerated; if the Trigger fires again, the block is withdrawn for the rest of the episode, since removing skills also removes their help. Scoring the whole response matters for a reasoning agent, which settles its action while reasoning: the action tokens alone barely change with the skills (§4).

We evaluate on the 134 unseen test games of ALFWorld (Shridhar et al., 2021) and on 300 questions of SRA-Bench ToolQA (Su et al., 2026; Zhuang et al., 2023), with Qwen3-8B and Qwen3-14B in thinking mode (Yang et al., 2025). TRIM wins 75, 71 and 65 games in three runs, on average 10.3 more than the retrieved skills (7.7 points, 95% CI $[2.0, 13.7]$; 19:4 games in its first run, $p = 0.003$); it also leads with Qwen3-14B and on ToolQA, and beats the same two stages applied at step 2 or at random steps. On development games, withdrawal helps where the signal fires repeatedly and hurts where it fires once, and at the states where TRIM withdraws, withdrawing beats keeping the edited skills. Our contributions are:

- A study of skill misfit with a reasoning agent: from the same states, the retrieved skills fail where withdrawing them succeeds (§3).
- TRIM, a training-free method in which the drafted response's likelihood with and without the skills decides when to revise and then withdraw them; to our knowledge, no prior method uses a model-internal signal to edit or withdraw an injected skill mid-episode (§4).
- Experiments on ALFWorld and ToolQA with two backbones, with ablations, and an analysis showing that withdrawal must be selective (§5–6).

2 Related Work

Agent skills. Agents keep experience as reusable skills, insights and workflows (Wang et al., 2024; Zhao et al., 2024; Wang et al., 2025b; Xia et al., 2026); packaged as agent skills in the prompt (Anthropic, 2025), curated skills raise average pass rates (Li et al., 2026c). Later work refines or grows skill libraries (Zhang et al., 2026c; Ouyang et al., 2026; Liu et al., 2026b; Ju et al., 2026; Wang et al., 2026c), retrieves skills for each plan step (Wang et al., 2026a), learns when to invoke them (Chen et al., 2026b) or which to inject (Li et al., 2026d), and adapts them to the backbone (Yu et al., 2026a) or into a task guide (Li et al., 2026b); most of these act between episodes or before the agent acts.

Skill misfit and adaptation. Skills can also cause failures an agent would not make without them (Dong et al., 2026): gains come with regressions (Tank and Nama, 2026), not every skill helps (Wang et al., 2026f), and gains shrink under realistic retrieval (Liu et al., 2026a) and larger libraries (Song and Wei, 2026); retrieved skills and memories can help in one state and mislead in another (Tu et al., 2026; Yuan et al., 2026; Ji et al., 2026). Our pairing (Figure 5; §3) adds a rerun-noise baseline and continuations from the same states with and without the skills. Closest to us, the concurrent EvoSkill-GUI revises its skills mid-rollout inside a multi-attempt loop (Chen et al., 2026a); other methods re-retrieve exemplars, select guidelines or revise plans during an episode (App. A).

Signals. Adaptive retrieval decides when to consult external knowledge from uncertainty and internal states (Jiang et al., 2023; Su et al., 2024), and concurrent work reads model internals to select skills (Lin et al., 2026) or to localize turns for offline skill revision (Zhang et al., 2026b). TRIM instead revises and, if the misfit persists, withdraws the skills within one attempt, with an unchanged executor: its trigger contrasts the likelihood of the drafted response, reasoning included, with and without the skills, the pointwise form of CXMI (Fernandes et al., 2021, 2023) that PECoRe thresholds (Sarti et al., 2024) (other uses: App. A), used only to trigger an intervention at inference time.

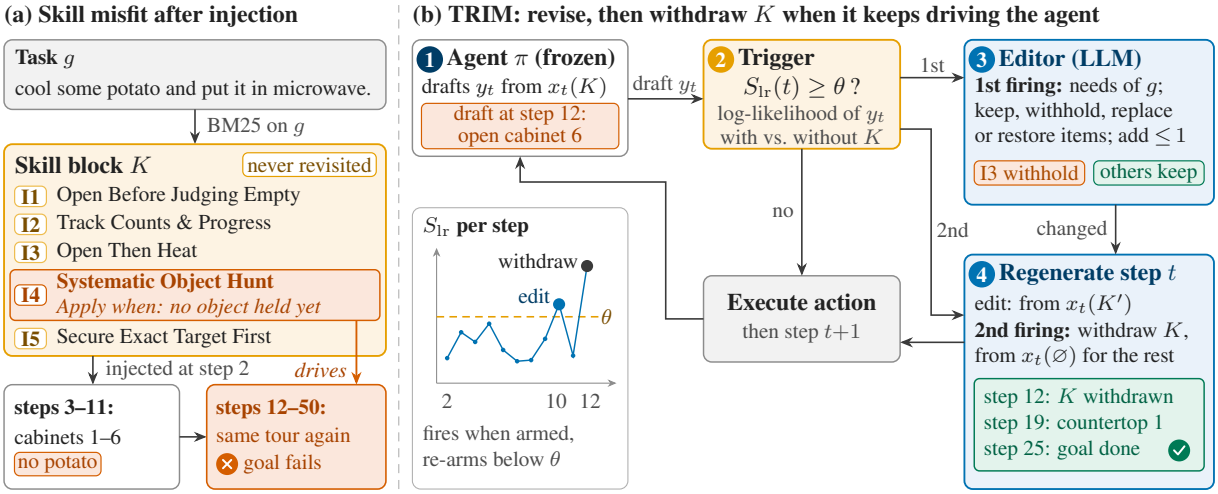


Figure 2: TRIM on one ALFWorld game (Qwen3-8B, thinking; App. F). (a) The block retrieved from the goal is never revisited: while no object is held, the search item I4 keeps applying, and the agent tours the cabinets and the fridge until step 50. (b) When the armed Trigger finds that the block drives the drafted response ($S_{\mathrm{lr}} \geq \theta$), the first firing sends the block to the Editor (step 10: the heating item I3 is withheld) and the second withdraws it (step 12); the step is regenerated, and without the block the agent reaches the countertop and wins.

3 Retrieved Skills Can Cause Failures

We compare the agent without the skill block (NoSkill) and with the block as retrieved (Skill), using Qwen3 in thinking mode on ALFWorld (§5.1); counts come from the first test run unless the development games are named.

Outcome changes. We pair NoSkill and Skill on each test game (Figure 5). With Qwen3-8B, the block raises wins only from 57 to 60 but changes the outcome of 45 games: it loses 21 of NoSkill's 57 wins and gains 24 games. With Qwen3-14B it raises wins from 61 to 63, losing 15 of NoSkill's wins and gaining 17. On the development games, where each arm ran twice, the block changes 35 of 140 outcomes, against 19 and 13 between two runs of NoSkill and of Skill, and it loses 14 of NoSkill's 64 wins, where a second NoSkill run loses 7. By task type, the block gains mostly on Clean (12 games against 2 lost) and loses on Cool (8 lost, 5 gained) and PickTwo (4 lost, none gained).

Same-state continuations. To hold the history fixed, we restored 12 states of development episodes with the block (six from games the skills harm, three from games they help and three from games that fail either way) and continued each three times with the block unchanged, with the block as revised by the editor of §4, and without the block (App. B). From the six harmed states, the unchanged block wins 0 of 18 continuations, the edited block 8 and no block 11; from the three helped states, 7, 6 and 6 of 9. The same state thus fails with the skills and succeeds without them, and withdrawing the block recovers more wins than editing it, at the cost of one win in nine where the skills help.

Failure patterns. Reading the 15 development games that the skills harm, we find two patterns (App. B). In nine, the agent never finds the target: 91% and 98% of its moves in the two Skill runs go to closed containers, against 59% and 63% without skills, and in six its reasoning cites a search item ("Systematic Object Hunt") in 37 to 48 of 49 steps. In four, BM25 attached a procedure written for another task type through a shared word, such as a cooling item in a heating task whose destination is the fridge; across the development heating games whose block holds that item, Skill wins none of 8 in either run and NoSkill 2 and 4. The other two failures are unrelated to the skills.

Accumulation within an episode. A search item does its harm by repetition: in the game of Figure 2, Skill circles through six cabinets and the fridge until step 50 and still cites "Systematic Object Hunt" at step 21, whereas NoSkill checks the countertop at step 3 and wins in 9 steps. A procedure for another task acts once its state arrives: in a development game that asks to heat a tomato and put it in the fridge, the agent cools the tomato right after heating it at step 26 because "the 'Enforce Cooling Before Placement' takes precedence", heats it again at step 45 and cools it again at step 47. Nei-

ther failure is visible in the task description, and both build up over steps that the retrieved text keeps steering.

4 TRIM: Revise, then Withdraw

TRIM leaves the agent unchanged and controls only which skill block it sees: a *Trigger* (§4.2) scores how strongly the block drives the agent, and a *two-stage intervention* (§4.3) revises the block at the first firing and withdraws it at the second (Figure 2b, Algorithm 1).

4.1 Problem Setting

We consider an LLM agent π that solves a task, described by a goal g, by interacting with an environment. At each step t the agent receives an observation o_t and the set of admissible actions $\mathcal{A}_t$, and selects an action a_t; the episode succeeds ($R = 1$) if the goal state is reached within T steps and fails ($R = 0$) otherwise. We write $\tau_t = (o_0, a_1, o_1, \ldots, o_t)$ for the history up to step t.

Skills are retrieved once, from the goal alone. Given a skill library $\mathcal{L}$ and a relevance score r, the retriever returns the skill block

$$K = \underset{s \in \mathcal{L}}{\text{top-}k}\ r(g, s) = \{\kappa_1, \ldots, \kappa_k\}, \qquad (1)$$

whose items κ_j are short procedures, some stating when they apply. The block enters the prompt at step t_0, and K_t denotes the block in effect at step t, so that $K_t = K$ unless it is changed during the episode. The agent then generates a reasoning trace z_t followed by an action,

$$y_t = (z_t, a_t) \sim \pi\big(\cdot \mid x_t(K_t)\big), \qquad (2)$$

where $x_t(B)$ is the prompt built from g, the recent history, o_t and $\mathcal{A}_t$ with block B, and $x_t(\varnothing)$ is the same prompt without skills. In our experiments r is BM25, $k = 5$, $t_0 = 2$ and $T = 50$ (App. C).

Misfit after injection. Since K is fixed before any observation, whether it helps depends on the states the episode reaches. Let $V(\tau_t, B) = \Pr[R = 1 \mid \tau_t, B]$ be the probability of success when the episode continues from τ_t with block B in the prompt. We say that the block misfits at τ_t if some modification B' of K_t, possibly the empty block, attains $V(\tau_t, B') > V(\tau_t, K_t)$; a block that fits at t_0 can therefore misfit later (§3). As V cannot be observed during an episode, the goal is to decide from signals available to the agent itself when and how to change the block, setting $K_{t+1} \in \{K_t, E(K_t), \varnothing\}$, so that a misfitting block is changed, a helpful one is kept, and interventions remain rare.

Algorithm 1 TRIM in one episode. The first firing sends the block to the editor; a second firing withdraws it for the rest of the episode.

Require: goal g; block K (BM25 on g); agent π; editor E; θ
$armed \leftarrow$ **true**; $n \leftarrow 0$ ▷ firings so far
for $t = 1, \ldots, 50$ **do**
  $y_t \leftarrow \pi\big(x_t(K)\big)$ ▷ reasoning, then action
  if $t \geq t_0$ **and** $K \neq \varnothing$ **then**
    $s \leftarrow S_{\text{lr}}(t)$ ▷ Eq. (4), whole response
    if $armed$ **and** $s \geq \theta$ **then** ▷ trigger fires
      $armed \leftarrow$ **false**; $n \leftarrow n + 1$
      if $n = 1$ **then** ▷ stage 1: review
        $K \leftarrow E.\textsc{Review}(K, g, \text{episode}, y_t)$
      else ▷ stage 2: withdraw
        $K \leftarrow \varnothing$
      if K changed **then** $y_t \leftarrow \pi\big(x_t(K)\big)$
    else if not $armed$ **and** $s < \theta$ **then**
      $armed \leftarrow$ **true** ▷ re-arm
  execute the action in y_t; **stop** if the episode ends

4.2 Trigger: Skill Influence on the Response

The Trigger asks whether the block, rather than the state, drives what the agent is about to do. For the drafted response y_t it compares each token's log-probability with and without the block,

$$c_{t,i} = \log \frac{\pi\big(y_{t,i} \mid x_t(K_t),\ y_{t,<i}\big)}{\pi\big(y_{t,i} \mid x_t(\varnothing),\ y_{t,<i}\big)}, \qquad (3)$$

the pointwise cross-mutual information (P-CXMI) of token $y_{t,i}$ with the block as context (Fernandes et al., 2021, 2023), and averages it over the whole response, reasoning included:

$$S_{\text{lr}}(t) = \frac{1}{|y_t|} \sum_{i=1}^{|y_t|} c_{t,i}. \qquad (4)$$

Since y_t is drawn from $\pi(\cdot \mid x_t(K_t))$, the sum $|y_t|\, S_{\text{lr}}(t)$ is a one-sample estimate of $\mathrm{KL}\big(\pi(\cdot \mid x_t(K_t)) \,\|\, \pi(\cdot \mid x_t(\varnothing))\big)$, how far the block moves the agent's response distribution from that of the same agent without skills; S_{lr} normalizes it by the response length. A high S_{lr} means the agent is following the retrieved text; whether that is right is left to the intervention.

Why the whole response. Restricting Eq. (4) to the action tokens gives the action-level ratio used without thinking. In thinking mode the action is

settled in the reasoning that precedes it, so the action tokens carry almost no contrast: over the 4,278 scored steps of an observation run on the development games, the action-level ratio has median 0.00 and 90th percentile 0.001, against 0.36 and 0.80 over the whole response.

Computation. Both terms of Eq. (3) score the same tokens: the draft is prefilled once after each prompt, which adds two prefills per step and no generation. The score uses the block in effect, so after an edit it measures the revised block; a draft without an action is not scored.

Firing rule. A run of high scores should count as one event, so the Trigger carries an armed state α_t, with $\alpha_{t_0} = 1$:

$$f_t = \alpha_t \mathbb{I}\big[S_{\mathrm{lr}}(t) \geq \theta\big], \qquad \alpha_{t+1} = \mathbb{I}\big[S_{\mathrm{lr}}(t) < \theta\big], \tag{5}$$

where $f_t = 1$ is a firing; an unscored step leaves α unchanged. We write $n_t = \sum_{t' \leq t} f_{t'}$ for the number of firings so far.

Threshold. θ, the only numeric hyperparameter, is set on an observation run of the agent on the 140 development games (ALFWorld valid_seen; Skill arm, no intervention), where $N_e(\theta')$ is the number of firings of Eq. (5) in episode e at threshold θ'. Before any test run, we set $\theta = \min\{\theta' : \frac{1}{140}\sum_e N_e(\theta') \leq 1.5\} = 0.9387$, the smallest threshold at which the Trigger fires at most 1.5 times per episode on average; 5.9% of the scored steps reach it (App. C.3).

4.3 Two-Stage Intervention

The intervention escalates with the number of firings: the first firing gets the change that keeps most of the block, and the second, the change that removes it. With E the editor and f_t, n_t from Eq. (5), the block for step t becomes

$$K_t^+ = \begin{cases} E(K_t, c_t) & f_t = 1,\ n_t = 1 \text{ (revise)}, \\ \varnothing & f_t = 1,\ n_t = 2 \text{ (withdraw)}, \\ K_t & \text{otherwise}, \end{cases} \tag{6}$$

where c_t is the editor's view of the episode (Stage 1 below), and $K_{t+1} = K_t^+$. If $K_t^+ \neq K_t$, step t is regenerated, $y_t \sim \pi(\cdot \mid x_t(K_t^+))$, with a new seed, and the regenerated action is executed. The second firing counts whether or not the first review changed the block, so an episode has at most one editor review, at most one withdrawal and at most two regenerated steps.

The order follows §3: from harmed states, no block recovers more continuations than a revised block (11 against 8 of 18), but where the skills help it loses one win in nine, and withdrawing at the first firing keeps only 10 of the 20 development games that need the skills (§5.3). TRIM therefore first revises, which can drop a misfitting item and keep the useful ones, and withdraws only if the revised block still drives the agent at a later state.

Stage 1: Editor. At the first firing, the editor reviews the block item by item in two calls; it is the agent's own checkpoint with thinking off and greedy decoding, so TRIM needs no second model.

Needs from the task. The first call sees only the task sentence and writes NEEDS, $N = E_{\mathrm{need}}(g)$: the outcome the task asks for, every object, place or value it names and every property it attributes to an object, each as a state the agent must produce. N is then fixed. Writing it before the episode is shown anchors the verdicts to the task rather than to the agent's current plan, which may itself follow the misfitting item.

Verdicts. The second call reads the context

$$c_t = (g, H, o_0, \tau_{t-8:t}, \mathcal{A}_t, \hat{a}_t, z_t^{[-1500:]}, K_t, N), \tag{7}$$

that is, the task, three harness facts H, the initial observation, the last eight executed steps, the admissible actions, the drafted action $\hat{a}_t$, the last 1,500 characters of the draft's reasoning, which show the item the agent leans on, the line-numbered block and N. For every item the editor gives a verdict $v_j \in \{\textsc{keep}, \textsc{withhold}, \textsc{replace}(r_j)\}$, and it may add one instruction u for a step that N requires, no item covers and the episode shows the agent missing. The edited block is

$$E(K_t, c_t) = \big[\rho(\kappa_j, v_j)\big]_{j:\, v_j \neq \textsc{withhold}} \oplus u, \tag{8}$$

with $\rho(\kappa, \textsc{keep}) = \kappa$, $\rho(\kappa, \textsc{replace}(r)) = r$ and $\oplus$ appending u if given.

Rules of the review. KEEP is the default, also when the editor is unsure. An item is withheld only if following it would make the agent do something N does not contain, or if its condition can never hold or is ruled out by the observations; a condition such as "Apply when" may describe a later moment. An item that serves N is never withheld; a replacement keeps the item's idea and corrects a detail, using only names that appear in the task, the observations, the admissible actions or the skill text; every change cites its evidence. The instructions

name no action vocabulary or task type, and the editor never sees S_{lr} or any outcome (App. C.4).

Stage 2: Withdrawal. At the second firing the editor is not called. The block is withdrawn for the rest of the episode, $K_{t'} = \varnothing$ for all $t' \geq t$, and step t is regenerated from $x_t(\varnothing)$. From then on every prompt is the no-skill agent's prompt in the same state: the agent continues from its current state, with the actions it took under the block in its history, and the next episode starts again with the retrieved block. The Trigger stops scoring, as there is no block to compare with, so each later step costs what it costs the no-skill agent.

Because the Trigger re-arms only below θ, a second firing marks a block that, after the review, again steers the agent at a later state. Rather than a second review, TRIM then removes the block: a search routine spans several items and plays out over many steps, which an item-level review can miss (§6), and removal recovered the most wins from harmed states.

5 Experiments

5.1 Experimental Setup

We evaluate on the 134 unseen games of ALFWorld (Shridhar et al., 2021; Côté et al., 2019) in the SkillRL harness (Xia et al., 2026) and on 300 questions of SRA-Bench ToolQA (Su et al., 2026; Zhuang et al., 2023), with Qwen3-8B and Qwen3-14B (Yang et al., 2025) in thinking mode; the block holds the five SkillBank skills ranked highest by BM25 (Robertson and Zaragoza, 2009). The baselines, NoSkill, Skill (the block as retrieved), *prompting* (a one-line instruction to adapt the skills), *SkillAligner* (Li et al., 2026b), *Planning* (Wang et al., 2026a) and *MASA* (Yu et al., 2026a), differ from TRIM only in which skills reach the prompt and how they are handled. We report success rates and compare arms per game with exact sign tests and 95% bootstrap intervals (details in App. D.1).

5.2 Main Results

Overall. With Qwen3-8B (Table 1), retrieved skills raise success from 42.5% to 44.8%, prompting reaches 50.0% and SkillAligner 48.9%, while TRIM reaches 56.0% in the run that shares the baselines' seeds and 52.5% over its three runs. Adapting the skills before the agent acts thus adds 5.2 and 4.1 points to the retrieved skills, whereas revising and withdrawing them during the episode

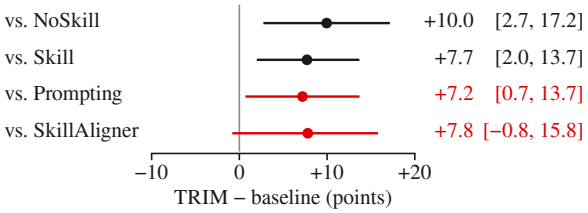


Figure 3: TRIM against each baseline on the 134 test games (Qwen3-8B, thinking): per-game success of TRIM averaged over its three runs (75, 71 and 65 wins) minus the baseline's mean over its runs (prompting and SkillAligner: three runs; NoSkill and Skill: one run), in points, with 95% bootstrap CIs over games (133 games for SkillAligner); run-1 tests: App. Table 9.

adds 11.2 points in that run. Its per-game success rate is 7.7 points above Skill's (95% CI $[2.0, 13.7]$) and 10.0 above NoSkill's ($[2.7, 17.2]$; Figure 3); its first run is ahead of Skill (19:4 games, $p = 0.003$) and NoSkill (28:10, $p = 0.005$) after a Holm adjustment (Holm, 1979).

Where the gains come from. The gains follow the misfit of §3. Where the retrieved skills hurt, TRIM recovers past the agent without skills: Cool rises from 52.4% with Skill to 76.2% (NoSkill 66.7%) and Pick from 37.5% to 58.3% (45.8%). Where they help, it keeps most of their gain on Clean (58.1%, against 61.3% with Skill and 29.0% without) and adds to it on Heat (56.5%, against 43.5% and 39.1%).

Other backbone and benchmark. With Qwen3-14B (three runs), the retrieved skills raise success from 43.5% to 49.5%, prompting reaches 49.3% and SkillAligner 51.0%, and TRIM 55.2%; on ToolQA TRIM answers 44.7% and 47.0% of the questions with the two backbones, ahead of every arm, and over three runs it beats prompting and SkillAligner with both backbones (pooled sign tests, $p \leq 0.003$; App. D.1).

Lost wins and cost. Keeping the skills loses 21 of NoSkill's 57 wins and dropping them loses 24 of Skill's 60; TRIM loses 10 and 4, the fewest of all arms with skills (Table 3). The adaptation baselines lose more: prompting loses 18 of NoSkill's wins and 6 of Skill's, and SkillAligner, which rewrites the skills before acting, 19 and 18. On the 26 games all these arms solve, TRIM takes 9.3 actions (Skill 8.2, NoSkill 11.5), and it generates 31.0k tokens per episode with editor calls, against 32.0k for Skill.

Method	ALFWorld: success (%)							SRA-Bench ToolQA: accuracy (%)						
	Pick	Heat	Cool	Clean	Look	PickTwo	All	Flt.	Cof.	Yelp	Abnb.	Agd.	DBLP	All
Qwen3-8B (thinking)														
Baselines														
NoSkill	45.8	39.1	66.7	29.0	50.0	29.4	42.5	14.0	38.0	48.0	22.0	16.0	20.0	26.3
Skill (BM25 top-5)	37.5	43.5	52.4	61.3	55.6	5.9	44.8	·	65.3	·	42.0	8.0	20.0	36.1 (169/300)
+ prompting	50.0	47.8	61.9	61.3	55.6	11.8	50.0	56.0	64.0	50.0	40.0	6.0	26.0	40.3
SkillAligner	70.8	22.7	47.6	58.1	50.0	35.3	48.9	50.0	40.0	48.8	32.0	8.0	10.0	31.1 (293/300)
Planning	45.8	39.1	52.4	58.1	50.0	17.6	45.5	40.0	50.0	53.6	36.0	8.0	20.0	33.1 (278/300)
MASA (released)	50.0	43.5	57.1	54.8	55.6	11.8	47.0	–	–	–	–	–	–	–
Ours														
TRIM	58.3	56.5	76.2	58.1	55.6	23.5	**56.0**	58.0	66.0	56.0	56.0	4.0	28.0	44.7
runs 1–3	58.3	50.7	68.3	53.8	55.6	21.6	52.5	–	–	–	–	–	–	–
Qwen3-14B (thinking)														
Baselines														
NoSkill	72.2	39.1	44.4	40.9	50.0	5.9	43.5	36.7	44.0	63.6	36.0	10.0	30.0	32.7 (260/300)
Skill (BM25 top-5)	68.1	27.5	49.2	64.5	68.5	5.9	49.5	54.0	66.0	46.0	34.0	10.0	42.0	42.0
+ prompting	68.1	30.4	50.8	60.2	70.4	3.9	49.3	52.1	70.0	100.0	28.0	8.0	20.0	35.7 (249/300)
SkillAligner	76.4	24.6	47.6	64.5	61.1	19.6	51.0	76.9	52.0	·	42.0	0.0	14.0	30.0 (213/300)
Planning	65.3	30.4	47.6	60.2	63.0	11.8	48.3	50.0	54.0	·	20.0	10.0	22.9	27.0 (200/300)
MASA (released)	66.7	31.9	49.2	61.3	64.8	13.7	49.8	–	–	–	–	–	–	–
Ours														
TRIM (runs 1–3)	69.4	40.6	58.7	65.6	68.5	17.6	**55.2**	62.0	72.0	52.0	44.0	8.0	44.0	47.0

Table 1: Main results with Qwen3 in thinking mode. ALFWorld: success on the 134 valid_unseen games per task type (games: 24, 23, 21, 31, 18, 17; SkillAligner finished 133 with Qwen3-8B), in the run that shares the baselines' seeds; runs 1–3 and Qwen3-14B baselines: three-run means (Qwen3-8B prompting and SkillAligner: run 1 shown, three-run comparison in Figure 3). ToolQA: accuracy on 300 questions, 50 per domain (Flt. Flights, Cof. Coffee, Abnb. Airbnb, Agd. Agenda). Green: interim (2026-10-12 07:35 UTC = 16:35 KST; App. D.3). Red: values of the assumed scenario (Limitations). –: not defined. Controls: App. Table 8.

5.3 Ablation Study

Table 2 removes one component at a time on the development games (139 games both arms finished). Without the withdrawal, success falls from 52.5% to 49.3%; at the 42 states where TRIM withdrew, continuations without the block win 35.7% against 23.8% with it (+11.9 points, $[1.2, 22.6]$; Figure 4). Withdrawing at the first firing wins about as often (51.4%) but keeps only 10 of the 20 games that need the skills, against 15 of 19 for TRIM, so withdrawal must be selective (§6). Other variants are in App. D.1. TRIM also wins more test episodes than the same two stages at step 2 or at random steps (211 against 187 and 190 of 402, $p \leq 0.009$), and withdrawing at the signal's state helps more than at nearby or random states (App. D.8).

Variant	Won	%	TRIM:var.
TRIM (edit, then withdraw)	73/139	**52.5**	–
full policy			
Edit only	69/140	49.3	14:10
tests the withdrawal			
Withdraw at first firing	72/140	51.4	14:13
tests the edit before withdrawal			
Regenerate, then withdraw	68/140	48.6	13:8
tests the first edit			
Edit, then random withdrawal	68/140	48.6	16:10
tests the signal for the 2nd firing			
No intervention (Skill)	70/140	50.0	13:9
reference			

Table 2: Components of TRIM on the 140 development games (Qwen3-8B, thinking; one run each, same seeds). Won / %: of finished games. TRIM:var.: games only TRIM or only the variant wins, among the 139 both finished. Signal variants: App. Table 10.

6 How TRIM Intervenes

Counts come from the first test run unless more runs or the development games are named.

Firing statistics. TRIM fires in 81 of the 134 test episodes of run 1, first at a median step of 4 (interquartile range 2–10); 64 of its 81 reviews change the block, and 34 episodes reach a second firing and withdraw the block, at a median step of 15 (9–20). Over the three runs, the first firing comes at a median step of 4 in every run (App. E). In the game of Figure 2, the edit at step 10 withholds the heating item but keeps both search items, the signal fires again at step 12, and after the withdrawal the agent revisits a few containers, then checks the sink basin and the countertop, finds the potato and wins

	Lost of			Tokens
Arm	NoSkill	Skill	Steps	Gen. (k)
NoSkill	–	24	11.5	38.1
Skill	21	–	8.2	32.0
+ prompting	18	6	9.4	30.5
SkillAligner	19	18	9.3	35.8
MASA	16	13	9.1	31.9
TRIM	10	4	9.3	31.0

Table 3: Lost wins, length and generation (Qwen3-8B, thinking, test run 1). Lost: wins of NoSkill (57) or Skill (60) that the arm loses. Steps: mean actions on the 26 games all arms solve. Gen.: generated tokens per episode, with editor calls.

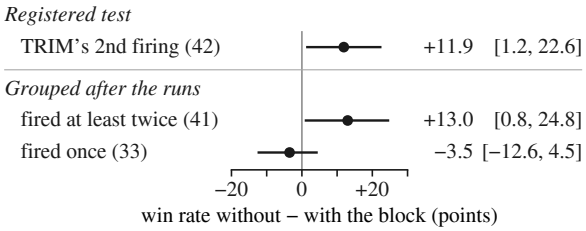


Figure 4: Withdrawing the block on restored development states (Qwen3-8B, thinking): win rate without the block minus with it, in points, with 95% bootstrap CIs. Top: the 42 states where TRIM withdrew, against keeping the edited block. Bottom: episodes grouped by whether the signal had fired at least twice or once by step 40, averaging three restored states each; the groups differ by 16.5 points, $[1.6, 32.0]$. Within an episode, withdrawing at the signal's state helps more than at the nearest or a random state without a firing (+9.5 and +10.1 points; App. Table 11).

at step 25 (Figure 2b).

Repeated firing and withdrawal. On the development observation run (no intervention), the rearmed Trigger fires 3.07 times per episode in the 15 games the skills harm, against 1.45 in the 20 games that need them and 0.59 in the 44 that every run wins; it fires at least twice in 11 of the harmed games, 8 of those that need the skills and 2 easy ones. To see whether withdrawing helps where the signal fires repeatedly, we restored 74 development episodes in which the signal fired, each at three of its states, and continued every state twice with the block and twice without it (App. D.8). In the 41 episodes where the signal fired at least twice, withdrawal raises the win rate by 13.0 points; in the 33 where it fired once, it lowers it by 3.5 points (difference 16.5 points, 95% CI $[1.6, 32.0]$; Figure 4).

Gains and losses of withdrawal. Split by how Skill and NoSkill fared on the game, withdrawal at the signal's state raises the win rate by 38.5 points in the games the skills harm and lowers it by 34.6 points in the games that need them (13 episodes each; App. Table 11), and in the same-state study it loses one win in nine where the skills help (§3). Removing the skills unconditionally would trade the games they harm for those they help: dropping them loses 24 of Skill's 60 test wins, whereas TRIM, which withdraws them only after a revision fails, loses 4 (Table 3).

Effect of each stage. The edit removes procedures written for other tasks: on the test games, its 81 first reviews withhold or replace 57 of the 79 items written for another task type, and on the development games, in 109 of 129 drafts that planned a procedure the task does not ask for, the regenerated reasoning no longer did. A search routine instead spans several items and plays out over many steps, and the withdrawal ends it: in the game of Figure 2 the agent reaches the countertop seven steps after the withdrawal, and in the game of Figure 7 it leaves the cabinets at the next step.

A matched episode. In one test game, TRIM's first 12 prompts equal Skill's while both agents search the cabinets; after the block is withdrawn at step 12, the agent leaves the cabinets, finds the mug and wins, whereas Skill, prompting, SkillAligner and the same intervention at step 2 stay among the cabinets for 50 steps (App. F, Figure 7).

7 Conclusion

A skill that fits the goal can still misfit the episode: with a reasoning agent on ALFWorld, the retrieved skills alone can make the agent fail from states where it would otherwise succeed, mostly by locking it into a search routine or another task's procedure. TRIM revises the injected skills when they drive the agent's drafted response and withdraws them if they keep driving it. In three test runs with Qwen3-8B, it wins 70.3 of 134 games on average, 10.3 more than the retrieved skills, and withdrawal helps where the signal fires repeatedly but hurts where it fires once, so skills should be withdrawn selectively rather than dropped. It also leads with Qwen3-14B and on ToolQA. Separating the misfits that skills cause from failures that happen anyway remains open. Skills should be revisited during execution, where they act.

Limitations

We evaluate TRIM with one model family, Qwen3, in thinking mode, on ALFWorld with one retrieved skill library and on SRA-Bench ToolQA; other environments, skill libraries written by people and other model families remain to be tested. The Trigger reads prompt log-probabilities, which many closed APIs do not expose, and scoring the drafted response with and without the block adds two prefills per step, 8.5% of an episode's wall-clock time with Qwen3-8B (App. E). TRIM revises or withdraws the retrieved block but does not write new skills, so failures whose procedure is missing from the library are outside its scope; the editor is the agent's own checkpoint, and a stronger editor may revise blocks more precisely.

References

Anthropic. 2025. Equipping agents for the real world with Agent Skills. Anthropic Engineering Blog. Written by Barry Zhang, Keith Lazuka, and Mahesh Murag; published October 16, 2025.

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations*, pages 9112–9141.

Giovanni Bonetta, Matteo Merler, Davide Zago, Rossella Cancelliere, and Bernardo Magnini. 2026. Selective agent guidance via entropy: Learning autonomous policies from imperfect VLM teachers. *Preprint*, arXiv:2609.01567.

Bofan Chen, Boxuan Zhang, Fei Tang, Zhengxi Lu, Yong Du, Tongbo Chen, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. 2026a. Reflect, revise, reuse: Training-free skill evolution for GUI agents. *Preprint*, arXiv:2609.17653.

Chishui Chen, Jiaye Lin, Te Sun, Yi Yang, Junxi Wang, Cong Qin, Yangen Hu, Lu Pan, and Ke Zeng. 2026b. Skill or skip? Learning selective skill invocation in agentic tasks via dual-granularity preference learning. *Preprint*, arXiv:2606.00510.

Minghao Chen, Yihang Li, Yanting Yang, Shiyu Yu, Binbin Lin, and Xiaofei He. 2024. AutoManual: Constructing instruction manuals by LLM agents via interactive environmental learning. In *Advances in Neural Information Processing Systems*, volume 37, pages 589–631. Curran Associates, Inc.

Benjamin Cohen-Wang, Harshay Shah, Kristian Georgiev, and Aleksander Mądry. 2024. ContextCite: Attributing model generation to context. In *Advances in Neural Information Processing Systems*, volume 37, pages 95764–95807. Curran Associates, Inc.

Marc-Alexandre Côté, Ákos Kádár, Xingdi Yuan, Ben Kybartas, Tavian Barnes, Emery Fine, James Moore, Matthew Hausknecht, Layla El Asri, Mahmoud Adada, Wendy Tay, and Adam Trischler. 2019. TextWorld: A learning environment for text-based games. In *Computer Games*, volume 1017 of *Communications in Computer and Information Science*, pages 41–75. Springer. Computer Games Workshop (CGW) at IJCAI 2018.

Florin Cuconasu, Giovanni Trappolini, Federico Siciliano, Simone Filice, Cesare Campagnano, Yoelle Maarek, Nicola Tonellotto, and Fabrizio Silvestri. 2024. The power of noise: Redefining retrieval for RAG systems. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 719–729. Association for Computing Machinery.

Gen Dong, Yanjie Gao, Liqun Li, Tianyin Xu, Yu Hua, and Fan Yang. 2026. Agent skills can be harmful: An empirical study of skill-induced failures in LLM agents. *Preprint*, arXiv:2608.11888.

Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. 2018. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1383–1392, Melbourne, Australia. Association for Computational Linguistics.

Yiyang Feng, Biddut Sarker Bijoy, Niranjan Balasubramanian, and Jiawei Zhou. 2026. Break it down, pass it on: Cross-task skill transfer in LLM agents. *Preprint*, arXiv:2608.20274.

Patrick Fernandes, Kayo Yin, Emmy Liu, André Martins, and Graham Neubig. 2023. When does translation require context? A data-driven, multilingual exploration. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 606–626, Toronto, Canada. Association for Computational Linguistics.

Patrick Fernandes, Kayo Yin, Graham Neubig, and André F. T. Martins. 2021. Measuring and increasing context usage in context-aware machine translation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 6467–6478, Online. Association for Computational Linguistics.

Yao Fu, Dong-Ki Kim, Jaekyeom Kim, Sungryull Sohn, Lajanugen Logeswaran, Kyunghoon Bae, and Honglak Lee. 2024. AutoGuide: Automated generation and selection of context-aware guidelines for large language model agents. In *Advances in Neural Information Processing Systems*, volume 37, pages 119919–119948. Curran Associates, Inc.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024.

CRITIC: Large language models can self-correct with tool-interactive critiquing. In *The Twelfth International Conference on Learning Representations*.

Shuang Guo, Xiaobo Sui, Fei Jiang, Ruixin Li, Manxiang Li, Rong Li, and Ziyang Ou. 2026. Towards strategy-level RSI for skill-augmented agents: Learning when to reuse skills from execution feedback. *Preprint*, arXiv:2609.26863.

Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. 2026. SWE-Skills-Bench: Do agent skills actually help in real-world software engineering? *Preprint*, arXiv:2603.15401.

Sture Holm. 1979. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70.

Jinwei Hu, Yi Qi, Xinmiao Huang, Youcheng Sun, Yi Dong, and Xiaowei Huang. 2026. Skill use or skill theater? Evaluating the reasoning backroom in skill-augmented language agents. *Preprint*, arXiv:2607.27484.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large language models cannot self-correct reasoning yet. In *The Twelfth International Conference on Learning Representations*.

Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, Pierre Sermanet, Tomas Jackson, Noah Brown, Linda Luu, Sergey Levine, Karol Hausman, and Brian Ichter. 2023. Inner Monologue: Embodied reasoning through planning with language models. In *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 1769–1782. PMLR.

Brian Ichter, Anthony Brohan, Yevgen Chebotar, Chelsea Finn, Karol Hausman, Alexander Herzog, Daniel Ho, Julian Ibarz, Alex Irpan, Eric Jang, Ryan Julian, Dmitry Kalashnikov, Sergey Levine, Yao Lu, Carolina Parada, Kanishka Rao, Pierre Sermanet, Alexander T. Toshev, Vincent Vanhoucke, and 26 others. 2023. Do as I can, not as I say: Grounding language in robotic affordances. In *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 287–318. PMLR.

Changxiu Ji, Amy Lu, Qizheng Zhang, and Kunle Olukotun. 2026. Sentry: Learning to recover from LLM agent failures at test time. *Preprint*, arXiv:2610.02994.

Zhengbao Jiang, Frank Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7969–7992, Singapore. Association for Computational Linguistics.

Ruofei Ju, Xinrui Wang, Xin Ding, Yifan Yang, Hao Wu, Shiqi Jiang, Qianxi Zhang, Hao Wen, Xiangyu Li, Weijun Wang, Kun Li, Yunxin Liu, Haipeng Dai, Wei Wang, and Ting Cao. 2026. EmbodiSkill: Skill-aware reflection for self-evolving embodied agents. *Preprint*, arXiv:2605.10332.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, and 17 others. 2022. Language models (mostly) know what they know. *Preprint*, arXiv:2207.05221.

Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han, and Rui Zhang. 2024. When can LLMs actually correct their own mistakes? A critical survey of self-correction of LLMs. *Transactions of the Association for Computational Linguistics*, 12:1417–1440.

Philipp Koehn. 2004. Statistical significance tests for machine translation evaluation. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, pages 388–395, Barcelona, Spain. Association for Computational Linguistics.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626. Association for Computing Machinery.

Ling Li, Qiuyu Shen, Zheng Jiang, Qinwei Ma, Yuxuan Liu, and Zhidong Deng. 2026a. SkillCycle: Co-evolving agent policies and skill banks. *Preprint*, arXiv:2610.09430.

Qinfeng Li, Dalin He, Yuntai Bao, Ying Yang, Ruoxi Chen, Xinyan Yu, Lizhou Liang, Ge Su, Wenqi Zhang, and Xuhong Zhang. 2026b. SkillAligner: Treating retrieved skills as adaptable drafts at execution time. *Preprint*, arXiv:2608.06880.

Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, and 59 others. 2026c. SkillsBench: Benchmarking how well agent skills work across diverse tasks. *Preprint*, arXiv:2602.12670.

Yanchao Li, Wanhao Liu, Ben Gao, Jiaqing Xie, Zhehong Ai, Na Zou, Yuqiang Li, and Tianfan Fu. 2026d. SkillsInjector: Dynamic skill context construction for LLM agents. *Preprint*, arXiv:2605.29794.

Bohan Lin, Hejia Geng, Xinyi Xie, Heng Zhou, Qinghua Xing, Bo Liu, Chen Zhang, and Yudong Zhang. 2026. Emotion2Skill: Model-internal emotion signals for adaptive skill selection and evolution. *Preprint*, arXiv:2608.09248.

Tianyu Liu, Jirui Qi, Paul He, Arianna Bisazza, Mrinmaya Sachan, and Ryan Cotterell. 2025. Pointwise mutual information as a performance gauge for retrieval-augmented generation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 1628–1647, Albuquerque, New Mexico. Association for Computational Linguistics.

Yujian Liu, Jiabao Ji, Li An, Tommi Jaakkola, Yang Zhang, and Shiyu Chang. 2026a. How well do agentic skills work in the wild: Benchmarking LLM skill usage in realistic settings. In *Third Conference on Language Modeling*.

Yuxuan Liu, Zhaochen Su, Lingyun Xie, Yuhao Zhang, Qing Zong, Jiahe Guo, Zhongwei Xie, Yiyan Ji, Yauwai Yim, Hongyu Luo, Xiyu Ren, Ruan Chenyu, Haoran Li, and Yangqiu Song. 2026b. SkillRevise: Improving LLM-authored agent skills via trace-conditioned skill revision. *Preprint*, arXiv:2606.01139.

Zhengxi Lu, Zhiyuan Yao, Zhuowen Han, Zi-Han Wang, Jinyang Wu, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. 2026. Self-distilled agentic reinforcement learning. *Preprint*, arXiv:2605.15155.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, pages 46534–46594. Curran Associates, Inc.

Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9802–9822, Toronto, Canada. Association for Computational Linguistics.

Viktor Moskvoretskii, Maria Marina, Mikhail Salnikov, Nikolay Ivanov, Sergey Pletenev, Daria Galimzianova, Nikita Krayko, Vasily Konovalov, Irina Nikishina, and Alexander Panchenko. 2025. Adaptive retrieval without self-knowledge? Bringing uncertainty back home. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6355–6384, Vienna, Austria. Association for Computational Linguistics.

Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. 2026. ReasoningBank: Scaling agent self-evolving with reasoning memory. In *The Fourteenth International Conference on Learning Representations*.

Tianjun Pan, Yuan Li, Hongda Wang, Linbo Jin, Mengfei Song, Lei Gao, Qiming Shi, Shaokang Fu, Jiarong Zhao, Chengyu Wang, and Chengfu Huo. 2026. Bidirectional context self-distillation for reinforcement learning of skill-based LLM agents. *Preprint*, arXiv:2608.09555.

Allen Z. Ren, Anushri Dixit, Alexandra Bodrova, Sumeet Singh, Stephen Tu, Noah Brown, Peng Xu, Leila Takayama, Fei Xia, Jake Varley, Zhenjia Xu, Dorsa Sadigh, Andy Zeng, and Anirudha Majumdar. 2023. Robots that ask for help: Uncertainty alignment for large language model planners. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 661–682. PMLR.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Gabriele Sarti, Grzegorz Chrupała, Malvina Nissim, and Arianna Bisazza. 2024. Quantifying the plausibility of context reliance in neural machine translation. In *The Twelfth International Conference on Learning Representations*, pages 25526–25554.

Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. 2023. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 31210–31227. PMLR.

Weijia Shi, Xiaochuang Han, Mike Lewis, Yulia Tsvetkov, Luke Zettlemoyer, and Wen-tau Yih. 2024. Trusting your evidence: Hallucinate less with context-aware decoding. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 2: Short Papers)*, pages 783–791, Mexico City, Mexico. Association for Computational Linguistics.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652. Curran Associates, Inc.

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2021. ALFWorld: Aligning text and

embodied environments for interactive learning. In *The Ninth International Conference on Learning Representations*.

Chan Hee Song, Jiaman Wu, Clayton Washington, Brian M. Sadler, Wei-Lun Chao, and Yu Su. 2023. LLM-Planner: Few-shot grounded planning for embodied agents with large language models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 2998–3009.

Hongwen Song and Song Wei. 2026. More skills, worse agents? Skill shadowing degrades performance when expanding skill libraries. *Preprint*, arXiv:2605.24050.

Weihang Su, Jianming Long, Qingyao Ai, Qiaozhi He, Yichen Tang, Changyue Wang, Yiteng Tu, Yingbo Wang, and Yiqun Liu. 2026. Skill retrieval augmentation for agentic AI. *Preprint*, arXiv:2604.24594.

Weihang Su, Yichen Tang, Qingyao Ai, Zhijing Wu, and Yiqun Liu. 2024. DRAGIN: Dynamic retrieval augmented generation based on the real-time information needs of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12991–13013, Bangkok, Thailand. Association for Computational Linguistics.

Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. 2024. Cognitive architectures for language agents. *Transactions on Machine Learning Research*.

Haotian Sun, Yuchen Zhuang, Lingkai Kong, Bo Dai, and Chao Zhang. 2023. AdaPlanner: Adaptive planning from feedback with language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 58202–58245. Curran Associates, Inc.

Darshan Tank and Baran Nama. 2026. The regression tax: Decomposing why skills help and hurt LLM agents. *Preprint*, arXiv:2607.22520.

Songjun Tu, Chengdong Xu, Qichao Zhang, Yiwen Ma, Yaocheng Zhang, Linjing Li, Dong Li, Xiangyuan Lan, and Dongbin Zhao. 2026. UCOB: Learning to utilize and evolve agentic skills via credit-aware on-policy bidirectional self-distillation. *Preprint*, arXiv:2606.29502.

Liam van der Poel, Ryan Cotterell, and Clara Meister. 2022. Mutual information alleviates hallucinations in abstractive summarization. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 5956–5965, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Chenxi Wang, Zhuoyun Yu, Xin Xie, Wuguannan Yao, Runnan Fang, Shuofei Qiao, Kexin Cao, Guozhou Zheng, Xiang Qi, Peng Zhang, and Shumin Deng. 2026a. SkillX: Automatically constructing skill knowledge bases for agents. *Preprint*, arXiv:2604.04804.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*.

Han Wang, Archiki Prasad, Elias Stengel-Eskin, and Mohit Bansal. 2025a. AdaCAD: Adaptively decoding to balance conflicts between contextual and parametric knowledge. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 11636–11652, Albuquerque, New Mexico. Association for Computational Linguistics.

Hao Wang, Guozhi Wang, Han Xiao, Yufeng Zhou, Yue Pan, Jichao Wang, Ke Xu, Yafei Wen, Xiaohu Ruan, Xiaoxin Chen, and Honggang Qi. 2026b. Skill-SD: Skill-conditioned self-distillation for multi-turn LLM agents. *Preprint*, arXiv:2604.10674.

Jiongxiao Wang, Qiaojing Yan, Yawei Wang, Yijun Tian, Soumya Smruti Mishra, Zhichao Xu, Megha Gandhi, Panpan Xu, and Lin Lee Cheong. 2026c. Reinforcement learning for self-improving agent with skill library. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1529–1550, San Diego, California, United States. Association for Computational Linguistics.

Mingxuan Wang, Guorun Yao, Fei Luo, Yinglong Guo, Chao Ning, Bo Wang, Hongyue Chen, Yanbiao Ma, and Jungong Han. 2026d. Memory control signals emerge before action in long horizon agents. *Preprint*, arXiv:2609.27286.

Tianxing Wang, Mingming Zhao, Shuai Huang, Huiyang Xu, Chaoyue Niu, Shengzhong Liu, and Fan Wu. 2026e. TROVE: Adaptive agent skill orchestration via trace-grounded route validation and editing. *Preprint*, arXiv:2609.05019.

Yixuan Wang, Yiyang Zhou, Yiming Liang, Congyu Zhang, Fuxiao Liu, Jiawei Zhou, and Huaxiu Yao. 2026f. Not all skills help: Measuring and repairing agent knowledge. *Preprint*, arXiv:2606.15390.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025b. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 63897–63911. PMLR.

Rong Wu, Daocheng Fu, Licheng Wen, Xuemeng Yang, Shu Zou, Jianbiao Mei, Yuxin Wang, Hairong Zhang, Yu Yang, Tao Hu, Cong Zhang, Botian Shi, and Pinlong Cai. 2026. MemHarness: Memory is reconstructed, not replayed. *Preprint*, arXiv:2607.28272.

Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. 2026. SkillRL: Evolving agents

via recursive skill-augmented reinforcement learning. *Preprint*, arXiv:2602.08234.

Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. 2024. Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs. In *The Twelfth International Conference on Learning Representations*, pages 23650–23678.

Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. 2026. How memory management impacts LLM agents: An empirical study of experience-following behavior. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 623–645, San Diego, California, United States. Association for Computational Linguistics.

Fangyuan Xu, Weijia Shi, and Eunsol Choi. 2024a. RECOMP: Improving retrieval-augmented LMs with context compression and selective augmentation. In *The Twelfth International Conference on Learning Representations*.

Rongwu Xu, Zehan Qi, Zhijiang Guo, Cunxiang Wang, Hongru Wang, Yue Zhang, and Wei Xu. 2024b. Knowledge conflicts for LLMs: A survey. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8541–8565, Miami, Florida, USA. Association for Computational Linguistics.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. Qwen3 technical report. *Preprint*, arXiv:2505.09388.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations*.

Shuo Yang, Jinyang Wu, Zhengxi Lu, Yuhao Shen, Fan Zhang, Lang Feng, Shuai Zhang, Haoran Luo, Zheng Lian, Zhengqi Wen, and Jianhua Tao. 2026. OPID: On-policy skill distillation for agentic reinforcement learning. *Preprint*, arXiv:2606.26790.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*.

Zijun Yao, Weijian Qi, Liangming Pan, Shulin Cao, Linmei Hu, Weichuan Liu, Lei Hou, and Juanzi Li. 2025. SeaKR: Self-aware knowledge retrieval for adaptive retrieval augmented generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27022–27043, Vienna, Austria. Association for Computational Linguistics.

Ori Yoran, Tomer Wolfson, Ori Ram, and Jonathan Berant. 2024. Making retrieval-augmented language models robust to irrelevant context. In *The Twelfth International Conference on Learning Representations*.

Jianxiang Yu, Jiapeng Zhu, Bochen Lin, Qier Cui, Zichen Ding, and Xiang Li. 2026a. Skill is not one-size-fits-all: Model-aware skill alignment for LLM agents. *Preprint*, arXiv:2605.30723.

Ziyang Yu, Yongliang Miao, Liang Zhao, Bowen Zhu, and Hasibul Haque. 2026b. SkillLens: Adaptive multi-granularity skill reuse for cost-efficient LLM agents. *Preprint*, arXiv:2605.08386.

Jiayi Yuan, Hao Li, Xinheng Ding, Wenya Xie, Yu-Jhe Li, Wentian Zhao, Kun Wan, Jing Shi, Xia Hu, and Zirui Liu. 2025. Understanding and mitigating numerical sources of nondeterminism in LLM inference. In *Advances in Neural Information Processing Systems*, volume 38, pages 169819–169851. Curran Associates, Inc.

Zhenhang Yuan, Shenghai Yuan, and Lihua Xie. 2026. RPMS: Enhancing LLM-based embodied planning through rule-augmented memory synergy. *Preprint*, arXiv:2603.17831.

Jiaxin Zhang, Prafulla Kumar Choubey, Kung-Hsiang Huang, Caiming Xiong, and Chien-Sheng Wu. 2026a. Agentic uncertainty quantification. *Preprint*, arXiv:2601.15703.

Kaixing Zhang, Changming Li, Yingdong Shi, Zheng Zhang, Kaitao Song, Wenjie Shi, Jingang Wang, and Kan Ren. 2026b. Rep2Skill: Representation-guided skill self-evolution for LLM agents. *Preprint*, arXiv:2609.39149.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Y. Zou, and Kunle Olukotun. 2026c. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations*.

Wenyuan Zhang, Xinghua Zhang, Haiyang Yu, Shuaiyi Nie, Bingli Wu, Juwei Yue, Tingwen Liu, and Yongbin Li. 2026d. ExpSeek: Self-triggered experience seeking for web agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 29255–29283, San Diego, California, United States. Association for Computational Linguistics.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17):19632–19642.

Ruiwen Zhou, Yingxuan Yang, Muning Wen, Ying Wen, Wenhao Wang, Chunling Xi, Guoqiang Xu, Yong Yu, and Weinan Zhang. 2024. TRAD: Enhancing LLM agents with step-wise thought retrieval and aligned decision. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 3–13. Association for Computing Machinery.

Yuchen Zhuang, Yue Yu, Kuan Wang, Haotian Sun, and Chao Zhang. 2023. ToolQA: A dataset for LLM question answering with external tools. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, volume 36, pages 50117–50143. Curran Associates, Inc.

A Extended Related Work

This section complements §2.

Context and revision during an episode. Skills need not suit another model (Yu et al., 2026a), and misaligned, irrelevant or conflicting context misleads agents and LLMs (Xiong et al., 2026; Shi et al., 2023; Xu et al., 2024b; Cuconasu et al., 2024). During an episode, exemplars are re-retrieved (Zhou et al., 2024), guidelines selected (Fu et al., 2024), retrieved experience rewritten by a concurrent trained policy (Wu et al., 2026) and plans revised (Sun et al., 2023; Wang et al., 2026e), and self-correction (Madaan et al., 2023; Gou et al., 2024; Shinn et al., 2023) often fails without external feedback (Huang et al., 2024; Kamoi et al., 2024). Adaptive retrieval also uses reflection tokens (Asai et al., 2024) or further internal-state triggers (Mallen et al., 2023; Yao et al., 2025); agents seek experience (Zhang et al., 2026d) or, concurrently, a teacher (Bonetta et al., 2026) when their entropy is high, and concurrent work reads model internals to time history compression (Wang et al., 2026d).

Skills and when they are adapted. Task-level induced skills often transfer below a no-memory baseline (Feng et al., 2026), and verifier-gated pipelines decide whether to reuse or rewrite skill units (Yu et al., 2026b). At injection, the concurrent SkillApt decides for each retrieved skill whether to load it, from evidence of past runs with and without it (Guo et al., 2026). Within an episode, LLM-Planner re-plans after failed actions (Song et al., 2023). These methods choose skills before acting or react to failures; TRIM keeps the executor unchanged and revises, then withdraws, the injected block when its likelihood contrast fires.

Filtering context. Retrieval pipelines compress or filter passages (Xu et al., 2024a), and filtering can discard relevant ones (Yoran et al., 2024); TRIM instead edits the guidance for the next action and can likewise withhold what the task needs (§6).

More on skills, plans and memories. Most public software-engineering skills leave pass rates unchanged (Han et al., 2026). Prompt optimizers refine instructions against a training score (Yang et al., 2024); cognitive architectures re-check rule preconditions at every decision cycle (Sumers et al., 2024); and planners check feasibility (Ichter et al., 2023) or use environment feedback (Huang et al., 2023).

Signals and on/off comparisons. Uncertainty signals often flag errors (Kadavath et al., 2022), and robot planners ask for help when uncertain (Ren et al., 2023); verbalized confidence is often too high (Xiong et al., 2024), and retrieved passages that are related but unhelpful lower accuracy more than random ones (Cuconasu et al., 2024).

Across adaptive-retrieval methods, simple uncertainty estimates often match more elaborate triggers (Moskvoretskii et al., 2025), and agents can reflect when their verbalized confidence calls for it (Zhang et al., 2026a). Such signals target an information need, whereas our trigger asks whether the skills drive the response. Concurrent work also argues that an agent's stated use of a skill does not reveal which decisions depend on it without an on/off comparison (Hu et al., 2026); our trigger makes that comparison for each drafted response until the block is withdrawn. The with/without-context contrast that our trigger uses also attributes generations to context (Cohen-Wang et al., 2024), reweights decoding (Shi et al., 2024; Wang et al., 2025a; van der Poel et al., 2022), selects prompts (Liu et al., 2025), trains skill-conditioned students (Wang et al., 2026b; Lu et al., 2026; Pan et al., 2026; Yang et al., 2026) and, concurrently, locates skill rules to revise between training cycles (Li et al., 2026a).

B Misfit Analysis Details

B.1 Same-State Study

We sampled 12 games of a development run (seed fixed in advance), stratified by its outcomes: six that Skill lost and NoSkill won (harmed), three that Skill won and NoSkill lost (helped) and three that both lost. In each, we restored the Skill episode at step k, one step after the agent first picked up

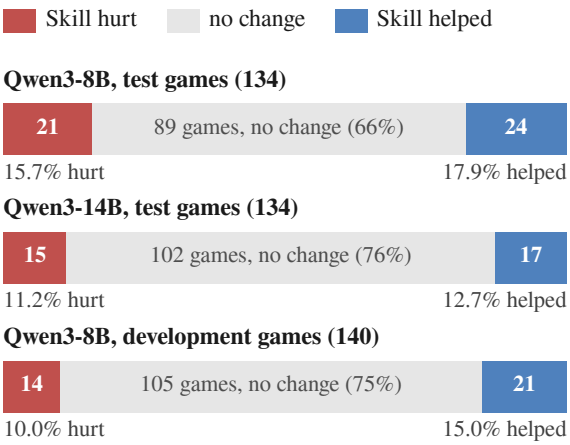


Figure 5: Where retrieved skills help and hurt (Qwen3 thinking, one run per arm). Hurt: NoSkill wins the game and Skill loses it; helped: the reverse; no change: both win or both lose. A second NoSkill run also changes 19 of the 140 development games.

State	Type	k	Keep	Edit	No block
Harmed (Skill lost, NoSkill won)					
S01	Heat	3	0/3	1/3	3/3
S02	Clean	3	0/3	2/3	1/3
S03	Cool	3	0/3	2/3	1/3
S04	Heat	30	0/3	1/3	1/3
S05	Heat	8	0/3	2/3	3/3
S06	Cool	3	0/3	0/3	2/3
Helped (Skill won, NoSkill lost)					
S07	Heat	45	2/3	2/3	0/3
S08	Clean	11	3/3	3/3	3/3
S09	Clean	6	2/3	1/3	3/3
Both lost					
S10–S12			0/9	0/9	0/9

Table 4: Same-state study on development states (Qwen3-8B, thinking): wins of three continuations per branch from the restored state at step k.

an object, or at step 3 if it never did, replaying the logged actions and checking that every restored prompt equals the logged one. From that state, each branch regenerates step k and continues to the 50-step limit three times (seeds fixed per branch), with no further intervention: A keeps the block, B uses the block as our editor revised it once (the first review of TRIM's editor, without the reasoning excerpt, which was added later), and N removes it (the no-skill prompt). A minimal editor allowed only to withhold one item kept every block and is not listed. Table 4 gives every state; four of the six harmed episodes never picked anything up, so their states are at step 3. Twelve states with three continuations each are a diagnostic, not a test.

B.2 Games the Skills Harm

Three analysts read the trajectories of the 15 development games that Skill lost in both of its runs while NoSkill won at least one. **Search tunnels (9 games).** In both Skill runs the agent never observed the target, and no Skill episode visited the place where NoSkill picked it up; 91% and 98% of Skill's moves went to closed containers, against 59% and 63% for NoSkill. In six games the reasoning cites "Systematic Object Hunt" in 37 to 48 of 49 steps; in the other three the item is rarely cited and NoSkill starts with the same tour, so the skills may not be the cause there. **Procedures for another task (4 games).** BM25 attached a state-change item for another task type through a shared word, such as "Enforce Cooling Before Placement" in heating tasks whose destination is the fridge, or "Open Then Heat" in cooling tasks whose destination is the microwave. Over all development games, heating games with the cooling item are won in 0 of 8 by both Skill runs and in 2 and 4 by NoSkill; cooling games with the heating item in 0 and 1 of 6 by Skill and 4 and 4 by NoSkill. **Unrelated (2 games).** One agent took a failed pick-up for a success, and one read "cool" as already cold, as NoSkill also did once.

What the editor does. Pooled over the development arms that only edit, the editor's 1,840 first-review verdicts are 998 keep, 732 withhold and 110 replace. It withholds most items written for other task types (410 of 574), but also half of the items for the task's own operation (276 of 551), 174 of them with a reason that denies the operation is needed, such as "already clean"; it almost never withholds generic search items (46 of 715) and kept "Open Before Judging Empty" in all 134 verdicts on it. 89% of the evidence for its first edits is the task sentence alone. Accepted edits changed the step's action in 9% of cases (43 of 477), below the 17% at which resampling alone changes an action, but they changed the plan: in 109 of the 129 drafts that planned an operation of another task type, the regenerated reasoning no longer did. On the test games, TRIM's editor (which sees the reasoning excerpt) withholds or replaces, in its 81 first reviews, 57 of the 79 items written for another task type, 26 of the 86 items for the task's own operation and 7 of the 49 search items; 64 of the 81 reviews change the block.

B.3 Run-to-Run Noise

With the same prompts and seeds, batched serving makes runs diverge: 19–28% of games differ in the first step, and each step before an edit diverges with probability 0.17. Without any edit, 9.4% of outcomes (13 of 139 games) differ between two Skill runs, and the difference in wins between two runs of 140 games has a standard deviation of about 3.7. Detecting a gain of 5 games in 140 with power 0.8 would need about 920 paired games, i.e., about seven repeats of the 140 games.

C Method Details

This appendix documents TRIM as it ran in all thinking-mode test runs; every setting was fixed on the 140 valid_seen games before any test run.

C.1 Agent Harness and Retrieval

Harness. We port SkillRL's ALFWorld agent loop (Xia et al., 2026) string for string: prompt templates, history format (the last two observation–action pairs), admissible-action list (without `help`) and action parser. Step 1 uses SkillRL's no-history template without skills; from step 2 on, the block appears under `## Retrieved Relevant Experience`. The parser takes the lower-cased text inside the first `<action>...</action>` of the answer, after the reasoning; if a tag is missing, the last 30 characters of the output are sent to the environment, which answers "Nothing happens.". An episode ends with success or after 50 actions.

Decoding. The agent is Qwen3-8B (Yang et al., 2025) with thinking enabled in its chat template, sampled with temperature 0.6, top-p 0.95 and top-k 20 (no penalties), at most 4,096 output tokens per response, and served with vLLM (Kwon et al., 2023) and its reasoning parser, which returns the reasoning and the answer separately. Each request carries a seed derived from the game, the step and the request kind (draft, regeneration), so that all arms draw the same seed for the same game and step; batched serving can still make two requests with one seed differ.

Retrieval. The skill bank is SkillRL's released SkillBank with 44 retrievable skills (12 general, 32 task-specific; its 11 "common mistakes" form a fixed section and are not retrieved). Each skill is indexed as "title. principle. when_to_apply" and the query is the goal sentence. We rank with Okapi BM25 (Robertson and Zaragoza, 2009) (k_1=1.5, b=0.75, lower-cased alphanumeric tokens, no stemming or stop-word removal) and keep the top five. General skills are rendered as `- **title**: principle` under `### General Principles` and task skills as the same line plus an indented `_Apply when: ..._` line under `### Task-Relevant Skills`.

C.2 Computing S_{lr}

The drafted response is rebuilt as `<think>`, the reasoning, `</think>` and the answer, and tokenized once. Each of the two prompts, $x_t(K)$ and $x_t(\varnothing)$, is chat-templated with thinking enabled and followed by these token ids, and one prefill returns the log-probability of every response token (temperature 0, one generated token, top-5 prompt log-probabilities). S_{lr} is the mean difference over all response tokens, in nats per token. $x_t(\varnothing)$ is the no-skill agent's prompt in the same state; the skill-off term rescores the same tokens and never regenerates the response without the block. A draft without an action in its answer is not scored and cannot fire. The same prefills give the other candidate signals of App. C.3: the ratio over the action tokens only, over the reasoning only or over its last 64 tokens with the action, the mean Jensen–Shannon divergence of the top-5 next-token distributions, and the mean entropy difference.

C.3 Choice of the Signal and of θ

Before any intervention ran, an observation run of the Skill arm on the 140 valid_seen games logged every candidate signal at every step and never intervened (4,278 scored steps). Over the whole response, S_{lr} has median 0.36 and 90th percentile 0.80; over the action tokens alone, 0.00 and 0.001. The rule, written before the selection ran, oriented each candidate by its strict AUROC over 138 finished games (the episode's maximum value; positives are the 8 games that Skill lost in both of its runs and NoSkill won in both of its runs), set each candidate's threshold to the smallest value at which the re-armed trigger fires at most 1.5 times per episode on the observation run, and sent the best candidate of each family to the development check. The whole-response ratio separated best among the single signals (AUROC 0.80, 95% bootstrap interval $[0.67, 0.92]$; Jensen–Shannon 0.75, entropy difference 0.72, last-64-token window 0.70); its threshold is $\theta = 0.938709$ (0.9387 in the text), which 5.9% of the scored steps reach, and at it the trigger fires in all 8 positive games and in 60% of

the others. Two combinations (a standardized mean of three signals, 0.75, and a two-stage gate, 0.82) went to the development check as well; all intervals overlap. Which signal and which intervention form TRIM was then decided by development success (App. D.7).

C.4 Editor

Model and call. The editor is the agent's checkpoint, Qwen3-8B, called with thinking disabled, temperature 0, seed 0 and at most 1,200 output tokens, once per episode (at the first firing). It first writes NEEDS, what the task sentence asks for, from the task sentence alone (a separate call that sees nothing else); NEEDS is then fixed.

Inputs and verdicts. Figure 6 shows the verdict prompt. It holds the task, three harness facts, the initial observation and the last eight executed steps (observations clipped to 300 characters), the admissible actions, the drafted action [P], the last 1,500 characters of the draft's reasoning, the line-numbered block, NEEDS and the list of items with their current state. The editor gives every item one verdict (keep, withhold, replace with a corrected instruction, or restore an earlier change) and may add one instruction. Withheld items disappear from the block that the agent sees, replaced items show the corrected instruction, and an added instruction is appended; if the block changed, step t is regenerated from $x_t(K')$. The editor never sees S_{lr} or any outcome.

Parser behavior. The parser reads an output line "`ADD: NONE | ...`" as an added instruction, so the agent's block then ends with a line such as "None required"; the test runs use the same parser as the development runs that selected the method.

C.5 Withdrawal

At the second firing the editor is not called: the block is removed for the rest of the episode, so that every later prompt is the no-skill agent's prompt in the same state, and step t is regenerated with a fresh seed. Scoring stops, because there is no block to compare with. The episode continues from its current state; the next episode starts again with the retrieved block.

Setting	Value
Block injected from	step $t_0 = 2$ (BM25 top five)
Signal	S_{lr} over the whole response
Threshold θ	0.938709 (re-armed below θ)
First firing	editor review; regenerate if changed
Second firing	withdraw the block; regenerate
Editor input	8 steps, [P], 1,500 chars of reasoning
Editor decoding	thinking off, temperature 0, seed 0
Agent decoding	thinking, T 0.6, top-p 0.95, top-k 20
Episode budget	50 actions

Table 5: Settings of TRIM, fixed on the development games.

D Experimental Details

D.1 Additional Setup Details and Ablations

Benchmarks. The test set is the 134 `valid_unseen` games of six task types (24 Pick, 23 Heat, 21 Cool, 31 Clean, 18 Look and 17 PickTwo), as in prior ALFWorld agents (Yao et al., 2023; Chen et al., 2024), with the SkillRL prompt and action parser (App. C). SkillRL's SkillBank holds 44 retrievable skills (12 general, 32 task-specific). SRA-Bench ToolQA has questions that need external tools in seven domains; we use 50 questions from each of six (SciREX, on which our earlier non-thinking runs answer at most 2% of the questions correctly, is left out; App. D.3).

Backbones. Qwen3-8B uses Qwen3's recommended sampling (temperature 0.6, top-p 0.95, top-k 20) and at most 4,096 tokens per response; each request's seed is derived from the game and step and shared by all arms. We use thinking mode because without it loops of repeated actions cause most failures and hide misfit: on the development games, the agent wins 30 and 37 of 140 games without and with skills when thinking is off, and 64 and 71 when it is on.

Baselines. The prompting baseline adds one line above the block (App. D.4); *SkillAligner* rewrites the five skills into a task guide before the agent acts (our port); *Planning* uses task-level plans to guide skill retrieval, and *MASA* swaps in the libraries released by Yu et al. (2026a). The arms differ only in which skills reach the prompt and how they are handled.

Metrics. Success is the share of games won within 50 actions; TRIM uses seed bases 1701–1703 and each baseline shares TRIM's first seed base. "a:b games" counts the games that only

```
You are checking the skill text given to an agent that acts in a text-based household environment
    (ALFWorld). The agent is in the middle of its episode.
## Task [G]
{GOAL}
## Harness facts
[H1] At every step the agent outputs one action inside <action></action>; only that action is executed.
    It should be one of the admissible actions listed for that step, otherwise the environment answers
    "Nothing happens." From step 2 on the agent sees the skill text below, its last 2 observation/action
    pairs and the current observation.
[H2] The episode succeeds as soon as the state described by the task [G] holds; nothing else is checked.
    The episode ends after 50 actions.
[H3] Objects and locations exist only if they appear in an observation or an admissible action, under
    those exact names (e.g. 'countertop 1', 'apple 2').
## Episode so far (actions and observations)
[O0] Initial observation: {...}   Action {k}: {...}   [O{k}] Observation {k}: {...}   (last 8 steps)
## Admissible actions at the current step [A]
{...}
## The agent's proposed next action (not executed yet) [P]
{DRAFTED ACTION}
## The agent's reasoning before [P] (end of it)
{LAST 1,500 CHARACTERS OF THE DRAFT'S REASONING}
## Skill text (line-numbered)
{S1:L1| ... }
## NEEDS (from the task sentence)
{NEEDS}
## Skill items (the state the agent currently sees)
I{k} = S1:L{a}-L{b} "{TITLE}": original | WITHHELD | REPLACED with: {...}
Added instruction in effect: none
## Your task
The skill text was retrieved for this task automatically and may not fit it. NEEDS below was written
    from the task sentence alone; use it as given.
Then give a verdict for EVERY skill item listed under 'Skill items', given what has been observed so far:
- KEEP: leave the item as the agent currently sees it (the default, also when unsure).
- WITHHOLD: following the item would make the agent do something NEEDS does not contain, or its condition
    can never hold here or is ruled out by the observations.
- REPLACE: the item's idea serves NEEDS but a detail is wrong for this task or episode; give the corrected
    instruction.
- RESTORE: an earlier change to this item was wrong; the agent should see the original item again.
If NEEDS requires a step or check that no item covers and the episode so far shows the agent missing it,
    give it as ADD (at most one).
- A condition in an item ('Apply when', 'after', 'once', 'before') may describe a LATER moment of the
    episode. Do not withhold an item only because its condition is not true yet; withhold it only if the
    condition can never hold for this task or the observations rule it out.
- Never withhold, and never leave withheld, an item that helps achieve something in NEEDS; if such an
    item is currently withheld, RESTORE it.
- Never RESTORE or ADD anything that asks for an action, object or change that NEEDS does not contain.
- Write instructions to the agent, not explanations. Use only actions, objects and locations that appear
    in the task, the observations, the admissible actions or the skill text.
- Every change must cite its evidence: [G] the task, [H1]/[H2]/[H3] a harness fact, [O<k>] observation k,
    [A] the admissible actions, [P] the proposed action.
Output exactly this and nothing else:
ITEMS:
- I<k> | KEEP, WITHHOLD, REPLACE or RESTORE | <evidence tag(s)> | <short reason, or the instruction for
    REPLACE; at most 25 words>
(one line per item, in order)
ADD: NONE, or ADD: <evidence tag(s)> | <instruction; at most 30 words>
```

Figure 6: The editor's verdict prompt at the first firing (template; blank lines removed). NEEDS comes from an earlier call that sees only the task sentence: "NEEDS = everything the task sentence [G] asks for: the outcome, every object, place or value it names, and every property it attributes to an object … a state the agent must PRODUCE, not a state that already holds."

one arm wins, tested with a two-sided exact sign test (Dror et al., 2018); effects are differences in per-game success, in percentage points, with 95% bootstrap intervals over games (Koehn, 2004), using TRIM's success rate per game over its three runs. Because batched serving changes outcomes between reruns even with fixed seeds (Yuan et al., 2025) (about 11% of development outcomes; App. B.3), we rely on per-game comparisons rather than on small differences in totals.

Main results by task type. TRIM gains most over the retrieved skills on Cool (76.2% against 52.4%) and Pick (58.3% against 37.5%); with Qwen3-14B, over three runs, SkillAligner is the strongest baseline (51.0%; per-run counts in Table 6), against 55.2% for TRIM (76, 73 and 73 wins; 56.7% in run 1). Over three runs TRIM wins 75, 71 and 65 games, on average 10.3 more than Skill's 60. With Qwen3-14B it is ahead of Skill in 40 games and behind in 20 (games one arm wins more often over the three runs; $p = 0.01$). Over three runs of each arm, it is ahead of prompting in 40:23 and 41:21 games with the two backbones (pooled, $p = 0.001$) and of SkillAligner in 48:26 and 38:24 (pooled, $p = 0.003$). On ToolQA it answers more questions than Skill (40:21 and 32:17 questions, $p = 0.02$ and 0.04).

Ablation details. One TRIM episode did not finish, so variants are compared on the 139 games both finished; the signals and steps compared while developing TRIM, which edit at every firing, are in App. D.7. Without the withdrawal (14:10 games for TRIM), the variant rescues 3 of the 15 games that the skills harm, against 9 (App. Table 10); the restored-state test continued each of the 42 states twice with the edited block kept and twice without it (App. D.8). Withdrawing at the first firing gives 14:13 games. Replacing the first edit with a plain regeneration of the step lowers success to 48.6% (13:8 games) and rescues 5 of the 15 harmed games, against 9. Withdrawing at a random step after the signal's edit, instead of at the second firing, lowers success to 48.6% (16:10 games) and rescues 4 harmed games, against 9. The whole-response ratio was chosen among five signals before these ablations; it separates the harmed games best (AUROC 0.80 against 0.70–0.75; App. C.3). The same intervention applied once at step 2 wins 46.5% of the test episodes over three runs, against 52.5% for TRIM (App. Table 8). The random-step control fires at each step with a fixed probability, set from TRIM's firing rate on the development games, and applies the same two stages. Over three runs TRIM wins 211 of 402 test episodes against 187 for PRE and 190 for RAND (34:15 and 35:12 games, $p = 0.009$ and 0.001) while firing 350 times against RAND's 380, and withdrawal at the signal's state beats withdrawal at the nearest or a random state without a firing (+9.5 and +10.1 points; Table 11).

Lost wins and length. Prompting loses 18 of NoSkill's and 6 of Skill's wins, and SkillAligner, which rewrites the skills before acting, 19 and 18. On the 26 games that all these arms solve, TRIM takes 9.3 actions against Skill's 8.2 and NoSkill's 11.5.

D.2 Runs and Servers

The test set is `valid_unseen` (134 games: Pick 24, Heat 23, Cool 21, Clean 31, Look 18, PickTwo 17) and the development set `valid_seen` (140 games); decoding and retrieval are as in App. C. The baselines (NoSkill, Skill, prompting and SkillAligner, with both backbones) ran on a local vLLM 0.8.5 server (two RTX A5000 GPUs, tensor parallel); TRIM, PRE, RAND, all development arms of TRIM and the same-state study ran on cloud A100 80GB GPUs with vLLM 0.29.0. The local server separates the reasoning with vLLM's `deepseek_r1` parser and the cloud server with its `qwen3` parser; both apply Qwen3's chat template with thinking enabled, so every comparison of TRIM with an ALFWorld baseline crosses the two servers. Both runners draw the first draft of each step with the same seed formula, so arms share seeds at the same game and step; neither server is batch-invariant. On the development games, the Skill arm wins 71 of 140 on the local server and 70 on the cloud server. Episodes that ended in an infrastructure error were re-run; one SkillAligner game (Qwen3-8B) failed in all three attempts and is excluded from that arm's counts and paired tests. TRIM and the two controls ran three times on the test games (seed bases 1701, 1702 and 1703), NoSkill and Skill with Qwen3-8B once (1701), prompting and SkillAligner with Qwen3-8B three times (1701–1703; wins 67, 58, 57 and 65 of 133, 57, 56), and each Qwen3-14B baseline three times with one configuration and server (Table 6); the Qwen3-14B Planning runs on ALFWorld use the same server, with the planning call also in thinking mode. TRIM with Qwen3-14B ran three times on the test games (seed bases 1701–1703), with θ set by the rule of §4.2 on an observation run of Qwen3-14B on the development games.

D.3 ToolQA in Thinking Mode

The ToolQA runs use 300 questions, 50 from each of six domains, one pass per arm, with the ReAct agent of SRA-Bench (at most 12 steps), Qwen3's thinking decoding and at most 4,096 tokens per step; every arm and both backbones run on cloud A100 80GB GPUs with vLLM 0.29 and its `qwen3` reasoning parser. Skill is the frozen top-5 BM25 block of each question; prompting, SkillAligner and Planning are as on ALFWorld, and Planning's plan call also runs in thinking mode. TRIM is unchanged except for θ: a ToolQA episode scores only 4.4 (Qwen3-8B) or 4.7 (Qwen3-14B) steps on average, so the rule of at most 1.5 firings per episode never binds; instead, θ keeps ALFWorld's share of firing steps, the smallest value reached by at most 5.891% of the scored steps of an observation run on 120 development questions disjoint from the 300 ($\theta = 0.770$ for Qwen3-8B and 0.758 for Qwen3-14B; 5.86% and 5.71% of the steps reach it). A question counts once its episode ends without an infrastructure error (a context overflow counts as wrong); infrastructure errors are re-run. The runs were still in progress at the snap-

Arm	Run	Pick	Heat	Cool	Clean	Look	Two	All
NoSkill	r1	16/24	9/23	11/21	15/31	10/18	0/17	61/134
	r2	18/24	8/23	10/21	12/31	8/18	1/17	57/134
	r3	18/24	10/23	7/21	11/31	9/18	2/17	57/134
Skill	r1	14/24	6/23	10/21	20/31	12/18	1/17	63/134
	r2	17/24	7/23	12/21	20/31	12/18	1/17	69/134
	r3	18/24	6/23	9/21	20/31	13/18	1/17	67/134
A6	r1	16/24	7/23	12/21	18/31	14/18	1/17	68/134
	r2	17/24	5/23	9/21	18/31	12/18	1/17	62/134
	r3	16/24	9/23	11/21	20/31	12/18	0/17	68/134
SkillAligner	r1	16/24	4/23	9/21	21/31	9/18	3/17	62/134
	r2	20/24	7/23	9/21	19/31	12/18	4/17	71/134
	r3	19/24	6/23	12/21	20/31	12/18	3/17	72/134
Planning	r1	16/24	6/13	13/19	19/30	12/16	0/4	66/106
	r2	·	·	·	·	·	·	·
	r3	·	·	·	·	·	·	·

Table 6: Qwen3-14B on the ALFWorld test games, wins / finished games per run (thinking mode). Green: run in progress (snapshot 2026-10-12 07:35 UTC = 16:35 KST); ·: no result yet.

Model	Arm	Flt.	Cof.	Yelp	Abnb.	Agd.	DBLP	All
Qwen3-8B	NoSkill	7/50	19/50	24/50	11/50	8/50	10/50	79/300
	Skill	·	32/49	·	21/50	4/50	4/20	61/169
	A6	28/50	32/50	25/50	20/50	3/50	13/50	121/300
	SkillAligner	25/50	20/50	21/43	16/50	4/50	5/50	91/293
	Planning	20/50	25/50	15/28	18/50	4/50	10/50	92/278
	TRIM	14/15	29/50	·	18/50	1/50	12/47	74/212
Qwen3-14B	NoSkill	18/49	22/50	7/11	18/50	5/50	15/50	85/260
	Skill	·	·	·	·	·	·	·
	A6	25/48	35/50	1/1	14/50	4/50	10/50	89/249
	SkillAligner	10/13	26/50	·	21/50	0/50	7/50	64/213
	Planning	1/2	27/50	·	10/50	5/50	11/48	54/200
	TRIM	·	37/48	·	10/50	4/50	4/14	55/162

Table 7: ToolQA in thinking mode, correct / finished questions per domain (one run, 50 per domain). Green: run in progress (snapshot 2026-10-12 07:35 UTC = 16:35 KST); ·: no result yet.

shot (2026-10-12 07:35 UTC = 16:35 KST); Table 7 gives the questions finished per domain, and a value turns black in Table 1 only when its run has counted all 300 questions and has finished its re-run pass.

D.4 Prompting Baseline

The prompting baseline puts one line directly above the BM25 block; the block itself is byte-identical to Skill. The line is adapted from a prompt used in a pilot study and made plural because the block holds several skills.

> **Prompting.** These skills were written without seeing your environment: treat them as a reference, adapt them to the tools and feedback you actually get, and still do any step the task needs even if the skills omit it:

D.5 Baseline Ports

SkillAligner (port). We re-implemented SkillAligner (Li et al., 2026b) from the prompts in its appendix; we did not use the authors' code. Before the episode, the agent's model makes one adaptation call (temperature 1 as in the paper, top-p 1, seed 0, at most 2,048 output tokens; the output cap and seed are our choices). Its input is the goal sentence, the five retrieved skills and, in the slot for the environment, ALFWorld's action format and step budget; it never sees the initial observation, the game identity or a solution. The output must be a JSON object with the string lists *Primary*, *Checks*, *Avoid* and *Fallback*. There is no repair or retry, and a schema failure counts as a failed episode (3 of 133 Qwen3-8B episodes). The guide replaces the raw block in the same slot from step 2 on and stays fixed.

Planning and MASA. Planning is our port of SkillX (Wang et al., 2026a): one planning call per episode sees only the goal, each plan step is a BM25 query over the same index, and the hits fill the same slot up to five skills. MASA (Yu et al., 2026a) edits skills offline per backbone; its released libraries for Qwen3-8B and Qwen3-14B extend the same SkillBank, so the baseline is a library swap with BM25 top-5. Both were run with thinking off in the non-thinking version of our experiments; in thinking mode, Planning and MASA ran on ALFWorld once with Qwen3-8B and three times with Qwen3-14B, Planning also runs on ToolQA with both backbones, and MASA released no ToolQA library.

Absolute success rates. SkillAligner reports, for Qwen3-8B on ALFWorld, 52.99% without skills, 61.94% with raw top-k skills and 80.60% with SkillAligner (Li et al., 2026b); in our harness the same backbone in thinking mode reaches 42.5%, 44.8% and 48.9% (our port). The harnesses differ: we use SkillRL's released ALFWorld agent loop (its prompt, two steps of history, at most 50 actions), whereas SkillAligner uses its own prompts and execution protocol. We therefore compare methods only within one harness and make no claim about absolute rates.

Not reproduced. SelSkill (Chen et al., 2026b) releases training and evaluation code but no trained selector checkpoint; reproducing it would require its GRPO initialization and three rounds of preference training, which falls outside our training-free comparison.

Arm	Pick	Heat	Cool	Clean	Look	Two	All	Runs
PRE	61.1	37.7	55.6	51.6	48.1	15.7	46.5	63, 63, 61
RAND	50.0	46.4	55.6	53.8	53.7	15.7	47.3	65, 62, 63
TRIM	58.3	50.7	68.3	53.8	55.6	21.6	52.5	75, 71, 65

Table 8: Success (%) over three test runs (402 episodes) of the same intervention at injection (PRE) and at random steps (RAND), and of TRIM, per task type (Qwen3-8B, thinking; Two: PickTwo). Runs: wins in runs 1, 2 and 3; PRE reviews at step 2, RAND fires with $p = 0.029$ per step.

TRIM	X:Y	p	Holm	Δ	95% CI
vs. PRE	22:10	0.050	0.200	+9.0	[0.7, 17.2]
vs. RAND	20:10	0.099	0.296	+7.5	[−0.7, 15.7]
vs. Skill	19:4	0.003	0.016	+11.2	[4.5, 17.9]
vs. NoSkill	28:10	0.005	0.025	+13.4	[4.5, 22.4]
vs. prompting	17:9	0.169	0.337	+6.0	[−1.5, 13.4]
vs. SkillAligner	29:19	0.193	0.337	+7.5	[−2.3, 17.3]
Runs 1–3 (X:Y: games one arm wins more often)					
vs. PRE	34:15	0.009	–	+6.0	[0.5, 11.4]
vs. RAND	35:12	0.001	–	+5.2	[0.2, 10.2]
Other pairs (X vs. Y)					
Skill vs. NoSkill	24:21	0.766	–	+2.2	[−7.5, 11.9]
RAND vs. PRE	15:13	0.851	–	+1.5	[−6.0, 9.0]

Table 9: Paired tests on the 134 test games (133 with SkillAligner; Qwen3-8B, test run 1 unless marked; runs 1–3 against the baselines: Figure 3). X:Y: games won by one arm only; p: exact sign test; Holm: over TRIM's six run-1 tests; Δ: success difference (points) with a bootstrap CI over games.

D.6 Same-Intervention Controls and Paired Tests

PRE and RAND share TRIM's runner, editor, regeneration and withdrawal rule and differ only in when they fire. PRE fires once, at step 2, so its second stage never occurs. RAND draws a seeded coin at every step from step 2 on, with $p = 0.0289$, TRIM's number of firings per scored step on the development games; its coin depends only on the game and step, never on the episode. Table 8 gives their success per task type and Table 9 every paired test of TRIM. Over the three runs, TRIM fires 350 times (0.87 per episode; a first firing in 246 episodes, a withdrawal in 104) and RAND 380 times (0.95; 238 and 142), with 211 and 190 wins of 402 episodes; PRE wins 187.

D.7 Development Runs and Selection

All development arms ran on the 140 valid_seen games with the same seeds as the no-edit observation run, and each is paired with it (Table 10). Strata come from two runs each of Skill and NoSkill, one of which is the observation run; comparisons with the observation run therefore regress to the mean (it wins none of the harmed games by definition, whereas episodes of other arms without any edit win 5 of 22 there). The first seven arms test signals and timings with edits only, at every firing, with an editor that does not yet see the reasoning excerpt. Their trajectories showed that the editor rarely changed the action and never removed the search items behind most harmed games (App. B), and the method was revised once within the same trigger: five candidates (the second group of Table 10) were compared by a rule registered before their results: highest development success, ties broken by fewer firings per episode. Edit-then-withdraw won the comparison (73 of 139; 0.521 even if its unfinished episode were lost, against at most 0.514 for every other candidate) and became TRIM; the random-timing version of the same intervention was run as its control. Withdrawing a third firing and a variant of the signal measured against the episode's own course were stopped part-way to save budget and are not used. Because TRIM was selected among these candidates on the same games, its development numbers are optimistic. After the choice, three component ablations were registered and run once each with the same seeds (last group of Table 10): one review without withdrawal, a plain regeneration of the step instead of the edit at the first firing, and the signal's edit followed by a withdrawal at a random step ($p = 0.0318$ per step, TRIM's rate of second firings on the development games).

D.8 Pre-registered Criteria and Component Tests

The signal, θ, the editor and the two controls were fixed on the development games, and the test games were run once (r1), with no change of method afterwards. Before any repeat result, we registered that TRIM counts as effective if its mean wins over three runs exceed Skill's 60 by five or more and it beats PRE in every run, the comparison with RAND being reported, not required: the first condition holds (mean 70.3) and so does the second (75, 71 and 65 wins against 63, 63 and 61), so TRIM meets it (Table 8).

Before their results we also registered three tests on the development games, each restoring logged episodes by replaying their actions (every restored prompt equals the logged one) and continuing them to the 50-step limit twice per branch with fixed

Arm	Won	Harmed	Needed	Fir.
No edit (Skill)	70/140	0/15	20/20	0.00
Edits only, at the steps of				
S_{lr}	66/140	3/15	15/20	1.46
JSD	62/140	4/15	12/20	1.09
Entropy	65/140	4/15	12/20	1.09
Combination	66/140	4/15	15/20	1.16
Two-stage gate	65/140	3/15	16/20	1.33
Random steps	70/140	5/15	17/20	1.59
Step 2	63/140	3/15	11/20	1.00
Revised interventions (S_{lr})				
Reasoning, $\leq$3 reviews	68/139	3/15	18/20	1.03
Edit, then withdraw	**73/139**	9/15	15/19	0.91
Withdraw at once	72/140	7/15	10/20	0.61
Attribution, one item	70/140	3/15	17/20	1.17
Arbitration	71/139	3/15	18/20	1.06
Two stages, random steps	69/140	5/15	16/20	1.10
Ablations of TRIM				
Edit only (one review)	69/140	3/15	14/20	0.61
Regenerate, then withdraw	68/140	5/15	15/20	0.92
Edit, random withdrawal	68/140	4/15	14/20	0.91

Table 10: Development arms (140 valid_seen games, Qwen3-8B, thinking). Harmed: games Skill lost in both of its runs and NoSkill won at least once; Needed: Skill won both, NoSkill at most one. Fir.: firings per episode. Bold: TRIM, chosen by the registered rule. Ablations: registered after the choice.

Comparison (development)	n	Δ	95% CI
Timing: d = win rate withdrawn − kept			
d(signal) − d(nearest)	74	+9.5	[0.7, 18.2]
d(signal) − d(random)	74	+10.1	[0.7, 19.6]
d: $\geq$2 firings − one firing*	41/33	+16.5	[1.6, 32.0]
d(signal) by stratum (episodes)			
harmed +38.5 (13), needed −34.6 (13), easy 0.0 (14), mixed +37.5 (8), fail either way +20.0 (25)			
Second firing: win rate of continuations			
withdraw − keep edited	42	+11.9	[1.2, 22.6]
withdraw − restore original	33	+4.5	[−9.1, 18.2]
restore original − keep edited	33	+9.1	[−3.0, 21.2]
First firing: win rate of continuations			
online edit − task-only edit	66	+0.8	[−3.8, 6.1]

Table 11: Component tests on restored development states (Qwen3-8B, thinking; two continuations per branch). Δ in points; bootstrap CIs over episodes or states. *Grouping examined after the runs.

Experiment	Question	Result
Repeated firing, held-out check (test)	does it mark where withdrawal helps?	[NaN]
Further runs against random steps (test)	fewer firings, higher success?	[NaN]
TRIM with Qwen3-14B (test)	other backbone	[NaN]
ToolQA in thinking mode, all arms	other benchmark	interim
Planning port on ALFWorld, Qwen3-14B (thinking)	baseline	running
MASA (thinking)	baseline	[NaN]

Table 12: Experiments of this version that are running or planned.

seeds and no further intervention (Table 11). *Timing*: in each of the 74 observation-run episodes in which the signal fired by step 40, three states are restored: the signal's (its second firing, or its only one), the nearest step without a firing and a random step without a firing; each continues with the block kept and with it withdrawn, and d is the win rate without the block minus that with it. *Second firing*: at the 42 states where TRIM withdrew the block, the continuations keep the edited block (E), withdraw it (N) or restore the original block (O, at the 33 states whose block the edit had changed). *First firing*: at TRIM's 84 first-firing states, the online edit is compared with an edit by the same editor that sees only the task sentence and the skills; the two edits differ at 66 states, whose continuations are compared. After the timing runs, we grouped their episodes by whether the signal had fired at least twice (§6); a check of this grouping on the test games, restoring every episode still running at step 10 and grouping it by its firings up to then, was registered before its observation run.

E Timing, Signal and Cost

When the arms intervene. Table 13 lists, for TRIM and the two controls, how often and when they fire and how the episodes end. Of TRIM's 81 reviews, 64 change the block and 17 leave it unchanged; a second firing follows in 34 episodes. RAND fires more often per episode (0.95 firings against TRIM's 0.86) and later, so its withdrawals fall in long episodes that are mostly lost by then. Over the three runs, TRIM's first firing has a median step of 4 in every run and its withdrawal of 15, 11.5 and 11.5, against 14, 12 and 15 and 27, 24 and 29 for RAND; TRIM fires 350 times in all and RAND 380. In 139 of TRIM's 191 lost episodes over the three runs, the agent never picks up the target (RAND: 157 of 212; PRE: 155 of 215).

Firings by development stratum. On the development observation run, the re-armed Trigger fires 3.07 times per episode in the 15 games the skills harm, 1.45 in the 20 that need them, 0.59 in the 44 easy games, 1.62 in the 13 mixed ones and 1.79 in the 47 that fail either way; it fires at least twice in 11, 8, 2, 5 and 19 of them.

	PRE	RAND	TRIM
Episodes with a firing	134	80	81
edits changing the block	125	79	64
withdrawn episodes	0	47	34
First firing: median [IQR]	2	14 [6, 28]	4 [2, 10]
Withdrawal: median [IQR]	–	27 [17, 41]	15 [9, 20]
Wins: no firing	–	46/54	27/53
firing, not withdrawn	63/134	12/33	29/47
withdrawn	–	7/47	19/34

Table 13: When the same intervention happens and how the episodes end (Qwen3-8B, thinking, test run 1, 134 games). IQR: interquartile range of the step.

The signal on the test games. Over TRIM's 3,384 scored test steps, S_{lr} has median 0.28 and 90th percentile 0.70, and 4.0% of the steps reach θ, against 0.36, 0.80 and 5.9% on the development observation run where θ was set; the test steps are fewer per episode because scoring stops after a withdrawal.

Cost. Scoring prefills the drafted response under both prompts at every step until a withdrawal: 79.6k prompt tokens per episode for TRIM, 2.6 times the 31.0k tokens it generates (Table 3). The prefills need no generation and took 8.5% of the episodes' wall-clock time; reading the skill-on log-probabilities at generation time would remove one of the two prefills, which we have not implemented. Editor calls add 1.4k tokens per episode. Wall-clock times are not comparable across our two servers, which ran at different concurrency.

F Cases

The cases come from the single test run with Qwen3-8B in thinking mode and were chosen after it; they illustrate mechanisms, not frequencies, and the supplementary material lists the file and line of every quoted step. All three wins below are cooling tasks in the same kitchen layout whose block holds the same search item; in none does the action at a firing step change, so the effect of an intervention shows only in later steps.

The game of Figure 2. Task: "cool some potato and put it in microwave." The block holds Open Before Judging Empty, Track Counts & Progress, Open Then Heat (a heating procedure), Systematic Object Hunt and Secure Exact Target First. With TRIM, S_{lr} stays between 0.31 and 0.84 at steps 2–9 while the agent searches the cabinets, and most drafts cite a skill item. At step 10 ("open cabinet 5", 1.12), the editor withholds Open Then Heat, giving the reason that it "hasn't happened yet", which its prompt forbids, and keeps the four other items. S_{lr} falls to 0.39 at step 11 and reaches 1.66 at step 12 ("open cabinet 6"), whose draft reads "according to the systematic object hunt skill, I should sweep every unvisited surface and open containers until the exact object is found"; the block is withdrawn. The agent revisits the fridge and three cabinets, goes to the sink basin at step 18 and to the countertop at step 19 ("Maybe the potato is on the countertop."), and wins at step 25. Skill shares the first six actions and then visits only the fridge and cabinets 1–6 until step 50; its reasoning cites skill items in 46 of 49 steps. NoSkill goes to the countertop at step 3 and wins at step 9; PRE makes the same edit at step 2 and loses; SkillAligner's guide turns the heating item into "Cool the potato with the microwave" and loses; prompting loses. RAND also wins this game (edit at step 5, withdrawal at step 21); the case shows how a withdrawal ends the search loop, and the withdrawal steps are compared over all episodes in App. D.8. The signal did not fire at step 3, where Skill and NoSkill first part (0.72).

An edit that keeps the misfitting item. Task: "put a cool tomato in microwave." At step 2, the first scored step, the draft reads "using the 'Open Then Heat' skill, I should open the microwave door, place the tomato inside, and heat it" ($S_{\text{lr}} = 1.32$). The editor's NEEDS misreads the task as heating ("the tomato is heated in the microwave"), and it keeps all five items. At step 5 the signal fires again (1.25; the draft cites the search item) and the block is withdrawn. The next six actions equal Skill's; at step 12 the agent goes to the countertop and at step 13 to the sink basin, where it finds the tomato, and it wins at step 25. Skill, PRE (the same all-keep review at step 2), RAND (edit at step 12, withdrawal at step 37), SkillAligner and prompting all lose. NoSkill wins at step 24.

The same history until the withdrawal (Figure 7). Task: "cool some mug and put it in coffeemachine." For steps 1–12, TRIM's prompts are identical to Skill's. At step 9 ($S_{\text{lr}} = 1.67$) the editor keeps all items, including a cleaning procedure whose condition cannot hold in a cooling task; at step 12 (1.16) the block is withdrawn. At step 13 the agent leaves the cabinets for the sink basin, finds the mug at step 18 and wins at step 23, whereas Skill, SkillAligner, prompting and PRE visit only cabinets 1–6 for 50 steps. NoSkill wins at step 32; RAND wins as well, after an edit at step

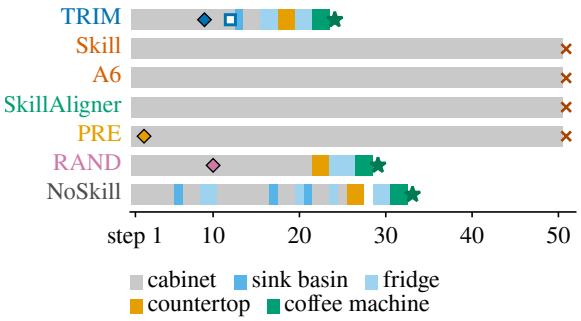


Figure 7: Where each arm goes in one test game, "cool some mug and put it in coffeemachine" (post hoc). Diamond: edit; square: withdrawal; star: goal done; cross: 50 steps used. TRIM's first 12 prompts equal Skill's.

10 that withholds the search item.

Games TRIM loses and Skill wins. All four diverge from Skill before TRIM's first firing, or never fire. (i) "put a clean soapbar in countertop": the editor withholds both cleaning items because the bar is "already clean per task"; after the withdrawal at step 16 the agent never cleans it. (ii) "clean some soapbar and put it in countertop": the editor withholds a helpful location item; the agent then loops through four cabinets for 46 steps with $S_{\mathrm{lr}} \leq 0.85$, so the block is never withdrawn. (iii) "cool some pan and put it in countertop": no firing until step 30; the agent finds the pan at step 16 but never cools it. (iv) "put a hot tomato in garbagecan": no firing at all ($S_{\mathrm{lr}} \leq 0.83$), although the drafts at steps 9, 10 and 13 cite a cooling item and the agent goes to the fridge at step 11, with S_{lr} between 0.24 and 0.36 at those steps.